

---

# Kalman Linear Attention: Parallel Bayesian Filtering For Efficient Language Modelling and State Tracking

---

Vaisakh Shaj Cameron Barker Aidan Scannell Andras Szecsenyi Elliot J. Crowley Amos Storkey

University of Edinburgh, United Kingdom

## Abstract

State-space language models such as Mamba and gated linear attention (GLA) offer linear-complexity, parallelisable alternatives to transformers, but their *linear* state updates limit expressivity and robust state tracking. We close this gap from a probabilistic angle, casting sequence mixing as exact Bayesian filtering with the Kalman filter as the core primitive. Classical Kalman filters give principled state and uncertainty estimates but are viewed as inherently sequential; we show that reparameterising them in *information form* turns their updates into an associative scan - so the per-token recurrent update is *non-linear* (a Möbius/precision recursion) yet remains *temporally parallel*. The resulting Kalman Linear Attention (KLA) layer is a drop-in sequence mixer that performs time-parallel probabilistic inference, carries an explicit belief-state uncertainty, and is strictly more expressive than GLA-style linear updates at the same computational cost. This expressivity translates directly into stronger state tracking: KLA solves permutation-composition ( $A_5$ ) tasks that linear SSMs and attention cannot, while staying scan-parallel. As a drop-in primitive it also matches or improves on modern SSMs and GLAs across synthetic token-manipulation and zero-shot commonsense benchmarks, and is among the first stacked Bayesian-filtering primitives trained at the billion-token scale.

## 1. Introduction

Scaling sequence modelling beyond quadratic attention (Vaswani et al., 2017) is a central goal in large language

Correspondence to: Vaisakh Shaj <vaisakhshaj@gmail.com>.

Proceedings of the 43<sup>rd</sup> International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

|                   | Softmax Attention  | SSMs / GLA       | KLA                      |
|-------------------|--------------------|------------------|--------------------------|
| Expressivity      | Nonlinear          | Linear           | Fractional lin. (Möbius) |
| Training eff.     | $\mathcal{O}(T^2)$ | $\mathcal{O}(T)$ | $\mathcal{O}(T)$         |
| Inference eff.    | $\mathcal{O}(T)$   | $\mathcal{O}(1)$ | $\mathcal{O}(1)$         |
| Seq. uncertainty  | ✗                  | ✗                | ✓                        |
| Parallel training | ✓                  | ✓                | ✓                        |

Table 1. High-level comparison of sequence-mixing primitives. KLA combines the efficiency of SSMs with nonlinear (Möbius) updates and explicit belief-state uncertainty.

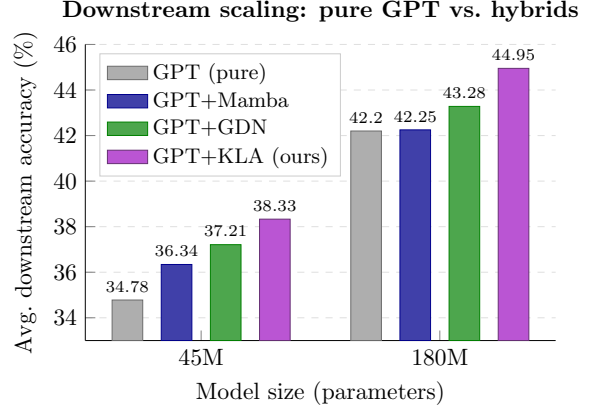
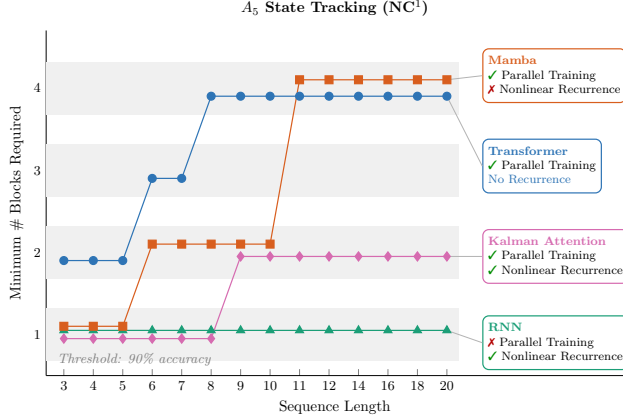
and foundation model research.<sup>1</sup> This has motivated interest in RNN-like architectures that support linear-time sequence modelling with efficient parallel training. Recent state-space models (SSMs), including S4/S5 (Gu et al., 2021; Smith et al., 2022), Mamba (Gu & Dao, 2023; Dao & Gu, 2024), and their successors, achieve this with  $\mathcal{O}(\log T)$  depth,  $\mathcal{O}(T)$  work, and linear or sublinear memory. This is essential for handling long contexts, on-device deployment and energy efficiency.

Despite their efficiency, adoption of SSMs is not as widespread as attention-based transformers. Recent work reinterprets modern SSMs and linear RNNs, including Mamba, as gated variants of linear attention (GLA) (Yang et al., 2023), where performance largely depends on how gates are defined. In all of these, the underlying hidden state updates are linear or affine, which limits their expressivity compared to softmax attention, whose normalisation induces nonlinear interactions between tokens. Furthermore, none of these represent state uncertainty explicitly. This contrasts with longstanding probabilistic linear state-space formalisms whose Kalman-filter updates carry explicit state uncertainty and update it through nonlinear precision recursions.

In this paper, we ask: *Can we overcome the linear update constraints common to current GLA models to develop a state-space block that efficiently implements exact Kalman filter updates?*

We introduce **Kalman Linear Attention (KLA)**, which formulates sequence modelling as a Bayesian filtering problem. KLA models two sources of uncertainty: process noise, which captures uncertainty in state evolution, and

<sup>1</sup>Code: [github.com/vaisakh-shaj/kalman-linear-attention](https://github.com/vaisakh-shaj/kalman-linear-attention)



**Figure 1. (a) Expressivity.** Minimum number of blocks required to solve the  $A_5$  (alternating group on 5 elements) permutation-composition task (Merrill et al., 2024). KLA’s fractional-linear (Möbius) updates fall between a fully nonlinear RNN and linear SSMs/transformers, solving the task at constant depth (1–2 blocks) where linear SSMs/attention require depth growing with sequence length, while remaining parallel-trainable. **(b) Downstream scaling.** Zero-shot accuracy (averaged over eight commonsense benchmarks) of softmax-attention GPT vs. GPT+SSM hybrids at 45M and 180M parameters. Replacing a single final attention layer with KLA (GPT+KLA) gives the strongest complement to attention.

observation noise, which captures uncertainty in the information provided by each input token. Crucially, this uncertainty is not merely an output: it directly controls how new information is gated in state spaces.

At first glance, such Bayesian updates seem inherently sequential. Our key insight is that the information form of the Kalman filter admits a fractional-linear / Möbius associative structure that can be implemented using parallel scan-based algorithms, *despite its nonlinear update computation*. This allows Kalman-style updates to achieve the same  $\mathcal{O}(\log T)$  parallel depth as models such as Mamba. The result is the **KLA** layer: a drop-in replacement for standard SSM or attention layers.

It is not enough to implement a new primitive without establishing that the extra expressiveness provides a benefit. We study the suitability of this continuous-state probabilistic model for discrete language modelling both theoretically and empirically, demonstrating important gains.

**Contributions.** Our contributions are as follows:

- C1 Associative reparameterisation of Kalman filtering:** we reparameterise the diagonal linear-Gaussian filter in information form, showing the precision recursion is a Möbius (fractional-linear) map that composes associatively, enabling parallel prefix scans.
- C2 Nonlinear gating from uncertainty:** the precision-ratio gates are history-dependent and nonlinear, going beyond linear/affine SSM and gated linear-attention updates while preserving linear-time scan structure.
- C3 Kalman Linear Attention layer:** we introduce KLA, a

drop-in sequence mixer for modern language-modelling pipelines that produces explicit belief-state uncertainty.

**C4 Scaling and empirical validation:** our scan-based implementation scales efficiently with sequence length and matches or outperforms modern SSMs and GLAs on state-tracking, associative recall, and zero-shot commonsense reasoning, and is among the first stacked Bayesian filters pretrained at the billion-token scale.

**Conflict of Interest Disclosure.** The authors declare no financial or other conflicts of interest; this is purely academic work.

## 2. Related Work

**Attention as filtering.** The idea that intelligent systems must filter information predates modern self-attention: in cognitive science, early theories framed attention as a mechanism that suppresses irrelevant signals and routes task-relevant ones under limited processing capacity (Broadbent, 1957; Treisman, 1969; Deutsch & Deutsch, 1963; Nielsen & Garcia, 2009). The core theme across these views is that attention allocates scarce processing by selecting what matters in context.

In modern language models, transformer self-attention can be viewed as a form of context-based filtering (Vaswani et al., 2017). Tokens compete via similarity and the softmax suppresses most inputs whilst allowing a small subset to dominate the update. But this selection is *exemplar-based*: it requires retaining and comparing against all past keys/values rather than compressing them into a fixed-size state, incurring  $\mathcal{O}(T^2)$  cost at long context lengths.

### Bayesian filtering and precision-control as attention.

Bayesian filters perform filtering as posterior inference: prior beliefs are updated with new observations weighted by their precision (inverse variance), so more reliable observations exert greater influence whilst unreliable evidence is down-weighted. This yields a principled, interpretable gating mechanism where selection follows from uncertainty propagation.

Precision-weighted prediction errors have also been proposed as a computational account of attention in neuroscience and predictive coding (Rao & Ballard, 1999; Feldman & Friston, 2010). In this view, attention modulation is based upon confidence in sensory signals as opposed to deterministic reweighting. We adopt this view and show that Bayesian filters can be integrated into modern sequence modelling.

### Sequence models as test-time in-context online learners.

Concurrently, recent sub-quadratic mixers have been unified as online learners that solve a per-token regularised regression in closed form, a view first articulated for SSMs by Longhorn (Liu et al., 2025; Wang et al., 2025) and extended in linear-attention and delta-rule variants (Yang et al., 2023; 2024; von Oswald et al., 2025; Hatamizadeh et al., 2026); we situate KLA within this view in Section 4.2 (with the full template in Section 4.3), where its posterior-mean update emerges as a gated RNN update with an adaptive learning rate from filtering.

**Temporal parallelisation of Bayesian filters.** Särkkä & García-Fernández (2020) pioneered temporal parallelisation of Bayesian filtering and smoothing using prefix-sum algorithms, requiring specialised parameterisations to ensure associativity. To achieve associativity, they construct it by lifting each filtering step into a 5-tuple augmented representation, in the linear case. We show that no such augmentation is needed: Kalman updates in the information form are already Möbius maps, so precision composes by  $2 \times 2$  matrix multiplication, a minimal, GPU-friendly structure (Theorems 1 and 2). Beyond parallelisation, we integrate these probabilistic primitives into learnable neural sequence models for language, where token-dependent observation likelihoods and dynamics are learned from data rather than specified a priori.

### Deep Kalman filters and probabilistic state-space models.

Kalman filtering (Kalman, 1960) has a long history in RL, control, and world modelling (Haarnoja et al., 2017; Watter et al., 2015; Hafner et al., 2019; 2020; Shaj Kumar et al., 2023). Deep Kalman filters embed linear-Gaussian SSMs in neural networks, trained by exact or variational inference (Krishnan et al., 2015; 2017; Karl et al., 2017; Fraccaro et al., 2016; Becker et al., 2019; Shaj et al., 2021), but their recurrent update limits scalability to long contexts. Recent deep-RL work pairs deterministic SSM backbones with

Kalman-style dynamics via Särkkä’s temporal parallelisation (Becker et al., 2024), or replaces them with parallelised Kalman layers for uncertainty under partial observability (Luis et al., 2024). In contrast, we adopt KLA as a *standalone*, closed-form probabilistic primitive for language modelling, competing directly with modern SSM/GLA mixers without an extra deterministic scaffold. Crucially, whereas these prior approaches typically employ a single Kalman layer, we hierarchically stack KLA across many layers, training deep multi-layer Bayesian-filtering architectures for language modelling at the billion-token scale.

## 3. Background and Preliminaries

Modern language models can be abstracted as the composition of two functions (Alonso et al., 2025): a shared representation (compression) map  $f(\cdot; \theta)$  and a task-specific head  $g$ . Given a prompt  $x_{1:T}$  of sequence length  $T$ , the backbone (the pretrained model) produces a compact latent representation  $z_{1:T} = f(x_{1:T}; \theta)$  that captures structure in the language; the head then maps  $z_{1:T}$  to outputs  $y_{1:T}$  (e.g., next-token prediction, tagging, regression). Concrete parameterisations for  $f$  balance expressivity with computational tractability: transformers use softmax attention as the core primitive, whereas state-space models (SSMs) such as Mamba adopt structured recurrence with linear complexity.

### Deterministic state-space models (SSMs) and Mamba.

Popular SSMs such as Mamba originate from structured state-space models (Gu et al., 2021). Building on their diagonal simplification (Gupta et al., 2022), a linear SSM maps a scalar input signal  $x(t)$  to an output  $y(t)$  through a latent state  $h(t) \in \mathbb{R}^N$ :

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t), \quad (1)$$

with  $x(t), y(t) \in \mathbb{R}$ ,  $\mathbf{A} = \text{diag}(a_1, \dots, a_N)$ ,  $\mathbf{B} \in \mathbb{R}^N$ , and  $\mathbf{C} \in \mathbb{R}^N$ . After discretisation, one processes a  $D$ -dimensional sequence  $\mathbf{X} \in \mathbb{R}^{T \times D}$  by running  $D$  independent copies of this recurrence, one per feature coordinate, or channel, each with its own scalar input  $x(t)$ .

**Mamba (S6)** (Gu & Dao, 2023) makes this recurrence input-dependent (selective). For a single channel the update is

$$\mathbf{h}_t = \bar{\mathbf{a}}_t \odot \mathbf{h}_{t-1} + \bar{\mathbf{b}}_t x_t, \quad y_t = \mathbf{c}_t^\top \mathbf{h}_t, \quad (2)$$

where  $x_t, y_t \in \mathbb{R}$ ; this system can be applied independently to each feature channel of a  $D$ -dimensional sequence. Here  $\mathbf{h}_t \in \mathbb{R}^N$  is the latent state,  $\bar{\mathbf{a}}_t, \bar{\mathbf{b}}_t \in \mathbb{R}^N$  are discretised diagonal dynamics, and  $\mathbf{c}_t \in \mathbb{R}^N$  is an input-dependent readout (the selective counterpart of  $\mathbf{C}$  in Equation (1)). The key property is *selectivity*: all parameters ( $\bar{\mathbf{a}}_t, \bar{\mathbf{b}}_t, \mathbf{c}_t$ ) are computed from the current input  $x_t$ , allowing the model to selectively filter which information to retain or discard at each step, while

preserving linear-time scanning. This input-dependent gating can be seen as a learned form of filtering; in Section 3 we formalise this intuition through Bayesian filtering with explicit uncertainty.

**State expansion.** The dimension  $N$  in Equation (2) is a design choice known as *state expansion* (Gupta et al., 2022; Poli et al., 2024; Yu & Erichson, 2025). With  $N=1$ , each feature channel maintains a single scalar recurrence and the state is  $\mathbf{h}_t \in \mathbb{R}^D$ . Setting  $N>1$  equips every channel with  $N$  parallel memory slots, expanding the state to  $\mathbf{H}_t \in \mathbb{R}^{N \times D}$ . Stacking the per-channel dynamics accordingly, we write  $\mathbf{A}_t, \mathbf{B}_t \in \mathbb{R}^{N \times D}$  and  $\mathbf{C}_t \in \mathbb{R}^{1 \times N}$ . How the dynamics are allocated across these slots is itself an algorithmic decision:  $\mathbf{A}_t$  and  $\mathbf{B}_t$  may be *distinct* per slot or *shared* (tied) across channels, depending on the model variant. State expansion trades a larger memory footprint ( $N \times D$  vs.  $D$ ) for richer per-channel recurrence histories, as each slot can capture a different effective time-scale.

**Attention from the SSM perspective.** Ignoring scaling constants, auto-regressive softmax self-attention computes

$$\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top + \mathbf{M})\mathbf{V}, \quad (3)$$

where  $\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}\mathbf{W}_q, \mathbf{X}\mathbf{W}_k, \mathbf{X}\mathbf{W}_v$  for  $\mathbf{X}, \mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times D}$ ,  $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{D \times D}$  and  $\mathbf{M} \in (-\infty, 1)^{T \times T}$  is an auto-regressive mask.

Unlike the linear recurrence in Equation (2), softmax attention maintains a distinct memory of each  $\mathbf{K}_t$  and  $\mathbf{V}_t$ , yielding  $\mathcal{O}(T^2)$  time and memory complexity.

By removing softmax we obtain Linear Attention  $\mathbf{y}_t = \mathbf{q}_t \sum_{i=0}^t \mathbf{k}_i^\top \mathbf{v}_i$  which permits an alternate recurrent form (Katharopoulos et al., 2020) that can be calculated in  $\mathcal{O}(T)$  time and  $\mathcal{O}(1)$  memory complexity, given by

$$\mathbf{H}_t = \mathbf{G}_t \odot \mathbf{H}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t \quad \text{and} \quad \mathbf{y}_t = \mathbf{q}_t \mathbf{H}_t. \quad (4)$$

where  $\mathbf{G}_t \in \mathbb{R}^{N \times D}$  is an optional gating term.

Identifying  $\mathbf{G}_t \equiv \bar{\mathbf{A}}_t$ ,  $\mathbf{k}_t^\top \equiv \bar{\mathbf{B}}_t$  and  $\mathbf{q}_t \equiv \mathbf{C}_t$  shows that GLA (Yang et al., 2023) matches Mamba when  $\mathbf{W}_v = \mathbf{I}$ .

**Gaussian state-space models and Bayesian filtering.** A classical probabilistic lens on sequential data is *state estimation*: we posit an (unobserved) latent state  $\mathbf{z}_t \in \mathbb{R}^d$  that evolves over time, and assume the observed sequence  $\mathbf{o}_t \in \mathbb{R}^p$  provides noisy evidence about that state. The goal of *Bayesian filtering* is to maintain the posterior belief  $p(\mathbf{z}_t \mid \mathbf{o}_{1:t})$  online, recursively combining a *dynamics prior* (predict) with an *observation likelihood* (update).

When the dynamics and observations are linear with Gaus-

sian noise, we obtain a linear-Gaussian state-space model:

$$\mathbf{z}_t = \mathbf{A}_t \mathbf{z}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_t), \quad (5)$$

$$\mathbf{o}_t = \mathbf{C}_t \mathbf{z}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \Sigma_t^{\text{obs}}), \quad (6)$$

in which the filtering posterior remains Gaussian,  $p(\mathbf{z}_t \mid \mathbf{o}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$  and can be computed in closed form via the *Kalman filter* (KF), which is optimal in the minimum mean-squared error (MMSE) sense under these assumptions.

**Control versus observation: two ways to read a token sequence.** This filtering view contrasts with the common “controlled dynamics” interpretation of deterministic mixers (e.g. SSM/CDE-style models) (Muca Cirone et al., 2024), where the input is an exogenous control driving the hidden state forward; Bayesian filtering instead treats it as noisy observations of a stochastic latent process. This swap, control versus observation, changes what is random and what computation is appropriate, forward simulation versus posterior inference (Figure 8). We adopt the filtering perspective throughout, yielding an attention-like mixer with explicit uncertainty propagation.

**Notational conventions.** We index token sequences by  $t \in \{1, \dots, T\}$ . Bold uppercase terms denote matrices (or collections over time), lowercase denotes scalars, and boldface denotes vectors/sequences (e.g.,  $\mathbf{x}_t$ ). In the diagonal (per-coordinate) parameterisation, we identify diagonal matrices with their diagonal terms and apply scalar recursions elementwise. An overbar denotes the discretised counterpart of a continuous-time parameter, e.g.  $\bar{a}_t$  is obtained by discretising a continuous-time decay parameter.

For completeness, Appendix C includes additional background material on related mathematical constructs such as Möbius (Fractional Linear) Transforms, the Information Form of Gaussian distributions, and Information Filters.

## 4. Method

We introduce *Kalman Linear Attention (KLA)* as a *probabilistic* sequence mixer. Instead of deterministically updating a hidden state as in modern SSM mixers, KLA maintains a belief state over a latent representation, consisting of a posterior mean and an explicit uncertainty (precision/covariance). This probabilistic view turns sequence mixing into state estimation from noisy token evidence, and provides a principled mechanism for adaptivity and a gating mechanism derived from relative uncertainties of state and observation.

Our method is organised around two ideas. **(i) A probabilistic SSM view of sequence mixing.** We cast token processing as inference in a linear-Gaussian state-space model. Each token contributes a (noisy) observation  $\mathbf{v}_t$  of the latent state through an observation operator  $\mathbf{k}_t$ , while the latent evolves



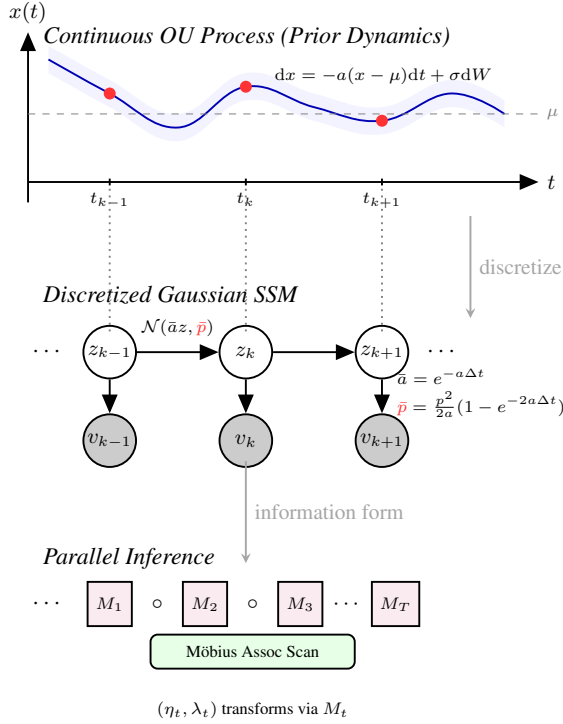


Figure 2. From OU dynamics to parallel inference. **(Top)** Continuous-time OU prior. **(Middle)** Discrete linear-Gaussian SSM. **(Bottom)** Möbius scan for parallel posterior state estimation.

under a continuous-time stochastic prior that we discretise for sequences.

**(ii) Parallelisable filtering via information form.** We address the main obstacle to using Bayesian filters as sequence mixers: classical filtering is recursive and appears inherently sequential. We show that by working in *information form* (precisions and natural parameters), the posterior updates acquire a parallelisation-friendly Möbius (fractional-linear) structure. This yields a scalable implementation of Bayesian sequence mixing with the computational profile of modern linear-time mixers, while preserving clear probabilistic semantics.

**Notation and scope.** Throughout this section, **blue terms** denote uncertainty-aware quantities absent in deterministic SSMs. For readability, the remainder of this section fixes the state-expansion factor to  $N=1$  and presents the *diagonal* (per-channel) Kalman filter, so that every recursion reduces to scalar (elementwise) operations. The state-expanded ( $N>1$ ) form - in which the state becomes a matrix  $\mathbf{H}_t \in \mathbb{R}^{N \times D}$  - is given in Table 3 and Algorithm 1. We stress that this matrix arises *solely* from state expansion (parallel memory slots per channel) and is *not* a full covariance: KLA’s covariance/precision remains diagonal throughout.<sup>2</sup>

<sup>2</sup>KLA uses two equivalent state parameterisations related by the precision:  $\eta_t, \mu_t$  are the per-channel ( $N=1$ ) *information-* and

#### 4.1. Stochastic Dynamics and Token Likelihood

**Dynamics prior (predict step) via OU discretisation.** Modern SSM mixers (Gu et al., 2021; Smith et al., 2022) are often motivated from continuous-time dynamics and then discretised to operate on token sequences. In this view, a simple linear ODE yields a discrete-time recurrence in which the discretised decay  $\bar{a}_t$  acts as a forget factor, and the step size  $\Delta t$  controls the effective time scale. This provides an appealing interpretation of SSM channels as a spectrum of memories parameterised by their decay rates.

KLA adopts the same continuous-to-discrete modelling pipeline, but makes one conceptual change: we treat latent evolution as inherently uncertain. Concretely, we specify a continuous-time *stochastic* dynamics prior and discretise it exactly. We use an Ornstein-Uhlenbeck (OU) process as the prior because it is the canonical mean-reverting diffusion (and the continuous-time analogue of a stable AR(1)): it preserves exponential forgetting while introducing an explicit process-noise term that captures unmodelled variability and uncertainty in the latent dynamics. Under exact discretisation, the OU prior induces a Gaussian transition

$$\mathbf{z}_t | \mathbf{z}_{t-1} \sim \mathcal{N}(\bar{\mathbf{a}}_t \odot \mathbf{z}_{t-1}, \bar{\mathbf{p}}_t), \quad (7)$$

$$\bar{\mathbf{a}}_t = e^{-\mathbf{a} \Delta t}, \quad \bar{\mathbf{p}}_t = \frac{\mathbf{p}^2}{2\mathbf{a}} \odot (1 - e^{-2\mathbf{a} \Delta t}), \quad (8)$$

where  $\bar{\mathbf{a}}_t$  denotes the decay and  $\bar{\mathbf{p}}_t$  the process-noise variance.

**Multi-channel specialisation.** Importantly,  $\bar{\mathbf{p}}_t$  is coupled to  $\bar{\mathbf{a}}_t$  by construction: the same decay and time-scale parameters  $\mathbf{a}$  that control decay also determine how uncertainty accumulates between observations. This yields per-channel specialisation along two linked axes: memory decay and drift. Different coordinates can learn different decay rates (short- vs. long-memory channels), and their process noise  $\bar{\mathbf{p}}_t$  sets how freely each channel can drift away from its predicted trajectory between tokens. This is different from Mamba, where the timescale/discretisation parameter is token-dependent and undertakes the role of selection/filtering at each timestep, whereas in our case selection/filtering is achieved purely via uncertainties without overloading  $\Delta t$  with additional roles.

#### Input token likelihood as noisy evidence (update step).

To connect this dynamics prior to input token sequences, we model each token as providing noisy evidence about the latent state:

$$\mathbf{v}_t | \mathbf{z}_t \sim \mathcal{N}(\mathbf{k}_t \odot \mathbf{z}_t, (\mathbf{\Lambda}_t^y)^{-1}), \quad (9)$$

*moment-form* means, and  $\mathbf{H}_t, \mathbf{S}_t \in \mathbb{R}^{N \times D}$  their state-expanded ( $N>1$ ) counterparts, related by  $\mathbf{H}_t = \mathbf{\Lambda}_t \odot \mathbf{S}_t$  (i.e.  $\eta_t = \lambda_t \odot \mu_t$ ). The parallel scan runs in the information form ( $\mathbf{H}_t$ , Algorithm 1); the moment form ( $\mathbf{S}_t$ , Table 3) is an equivalent representation used by the query readout. The explicit state-expanded recursions are given in Equations (22) and (23).

| Attention                                                          | Deterministic SSM / GLA                                       | KLA (probabilistic)                                                                                        |
|--------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Query operator $\mathbf{q}_t$                                      | Output/readout map $\mathbf{C}_t$                             | Query/readout operator $\mathbf{q}_t$                                                                      |
| Key operator $\mathbf{k}_t$                                        | Input to latent map $\mathbf{b}_t$                            | Observation model/operator $\mathbf{k}_t$                                                                  |
| Value $\mathbf{v}_t$                                               | Token input/control $\mathbf{u}_t$                            | Noisy observation $\mathbf{v}_t$                                                                           |
| Output $\mathbf{y}_t$                                              | Output $\mathbf{y}_t$                                         | Output $\mathbf{y}_t$                                                                                      |
| <i>Hidden state (implicit in attention, explicit in SSMs/KLA):</i> |                                                               |                                                                                                            |
| —                                                                  | Hidden state $\mathbf{h}_t$                                   | Posterior belief $\mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Lambda}_t^{-1})$ over $\mathbf{z}_t$        |
| —                                                                  | State decay $\bar{\mathbf{a}}_t = e^{-\mathbf{a}(t)\Delta t}$ | State decay $\bar{\mathbf{a}}_t = e^{-\mathbf{a}\Delta t}$                                                 |
| <i>Uncertainty terms (explicit in KLA only):</i>                   |                                                               |                                                                                                            |
| —                                                                  | —                                                             | Value precision $\boldsymbol{\Lambda}_t^v$                                                                 |
| —                                                                  | —                                                             | State precision $\boldsymbol{\Lambda}_t$                                                                   |
| —                                                                  | —                                                             | Process noise $\bar{\mathbf{p}}_t = \frac{\mathbf{p}^2}{2\mathbf{a}} \odot (1 - e^{-2\mathbf{a}\Delta t})$ |

Table 2. Notation alignment across attention, SSMs, and KLA. Precisions ( $\boldsymbol{\Lambda}$ ,  $\boldsymbol{\Lambda}^v$ ,  $\boldsymbol{\Lambda}^p$ ) are inverse variances encoding confidence: higher precision means lower uncertainty.

where  $\mathbf{v}_t$  is token-derived content,  $\mathbf{k}_t$  is an observation operator, and  $\boldsymbol{\Lambda}_t^v$  is the value precision representing confidence in the token evidence. Combining Equation (7) with Equation (9) yields a linear-Gaussian state-space model, and *sequence mixing corresponds to posterior inference* of the latent state given token evidence.

**Output readout as a query/task-conditioned projection of the belief state.** So far,  $(\mathbf{k}_t, \mathbf{v}_t, \boldsymbol{\Lambda}_t^v)$  define how input token evidence updates the latent belief  $p(\mathbf{z}_t | \mathbf{v}_{1:t})$ . A remaining design choice is how to use this belief to produce an output representation for downstream prediction. Deterministic SSM mixers typically apply a readout  $\mathbf{y}_t = \mathbf{C}_t \odot \mathbf{h}_t$ , while transformers use a query to determine what information should be extracted from stored context. KLA adopts an analogous view: the *query*  $\mathbf{q}_t$  specifies *what we want to read out* from the inferred belief state at time  $t$ . To make this explicit, we introduce a (linear-Gaussian) readout model

$$\mathbf{y}_t | \mathbf{z}_t \sim \mathcal{N}(\mathbf{q}_t \odot \mathbf{z}_t, (\boldsymbol{\Lambda}_t^{\text{out}})^{-1}), \quad (10)$$

where  $\boldsymbol{\Lambda}_t^{\text{out}}$  is an *output precision* (readout noise). In this work we take the deterministic-readout limit  $\boldsymbol{\Lambda}_t^{\text{out}} \rightarrow \infty$  (equivalently, zero readout noise), so that the output is the posterior mean projection

$$\mathbf{y}_t = \mathbf{q}_t \odot \boldsymbol{\mu}_t, \quad (11)$$

which mirrors the role of a query-conditioned readout in attention and a  $\mathbf{C}_t$ -projection in SSMs. An intuitive analogy is accumulating beliefs over time via posterior inference

through an input modality  $\mathbf{v}_t$  (say image sensors) via observation model  $\mathbf{k}_t$  and converting the latent beliefs to an output modality (say proprioceptive sensors)  $\mathbf{y}_t$  via query model  $\mathbf{q}_t$ .

**Gaussian SSM view of  $q$ - $k$ - $v$  interactions.** We can summarise the attention-aligned probabilistic interpretation as a linear-Gaussian state-space model in which keys and values parameterise the *likelihood* (token evidence), while queries parameterise a *readout* applied after inference. Let  $\mathbf{z}_t$  denote the latent state and let  $(\mathbf{k}_t, \mathbf{v}_t, \boldsymbol{\Lambda}_t^v)$  be token-dependent likelihood parameters produced from the input sequence. The generative model is:

$$\text{Prior} \quad \mathbf{z}_t | \mathbf{z}_{t-1} \sim \mathcal{N}(\bar{\mathbf{a}}_t \odot \mathbf{z}_{t-1}, \bar{\mathbf{p}}_t), \quad (12)$$

$$\text{Evidence} \quad \mathbf{v}_t | \mathbf{z}_t \sim \mathcal{N}(\mathbf{k}_t \odot \mathbf{z}_t, (\boldsymbol{\Lambda}_t^v)^{-1}), \quad (13)$$

$$\text{Readout} \quad \mathbf{y}_t | \mathbf{z}_t \sim \mathcal{N}(\mathbf{q}_t \odot \mathbf{z}_t, (\boldsymbol{\Lambda}_t^{\text{out}})^{-1}). \quad (14)$$

In this view,  $\mathbf{k}_t$  specifies the observation geometry (how token evidence constrains the latent),  $\mathbf{v}_t$  is the observed token evidence, and  $\boldsymbol{\Lambda}_t^v$  is its reliability (precision). The query  $\mathbf{q}_t$  instead specifies what component of the inferred latent state is exposed as the output.

## 4.2. Parallel Inference via Möbius and Affine Scans

A major obstacle to using Bayesian filters as sequence mixers is that classical filtering is *recursive* and thus appears inherently sequential. Our key observation is that, in *information form*, the updates admit a *compositional* structure that can be implemented with parallel scans.

**Information form.** We represent the Gaussian posterior by its precision and natural (canonical) parameter:

$$p(\mathbf{z}_t | \mathbf{v}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Lambda}_t^{-1}), \quad \boldsymbol{\eta}_t := \boldsymbol{\Lambda}_t \odot \boldsymbol{\mu}_t. \quad (15)$$

This parameterisation is convenient because, in linear-Gaussian models, incorporating token evidence corresponds to *adding* canonical parameters (precisions and information means), while the remaining “predict” transformation induced by the dynamics takes a structured form. As we show next, these structured updates *compose associatively* across time, which is exactly what enables parallel prefix scans.

**Theorem 1** (Precision Update as a Möbius Transformation). *Let  $\boldsymbol{\Lambda}_t$  be the posterior precision at time  $t$  in the diagonal linear-Gaussian model Equation (7)–Equation (9). Define  $\boldsymbol{\phi}_t := \mathbf{k}_t^2 \odot \boldsymbol{\Lambda}_t^v$ . Then the map  $\boldsymbol{\Lambda}_{t-1} \mapsto \boldsymbol{\Lambda}_t$  is a linear-fractional (Möbius) transform:*

$$\boldsymbol{\Lambda}_t = \mathbf{M}_t(\boldsymbol{\Lambda}_{t-1}) = \frac{\boldsymbol{\alpha}_t \odot \boldsymbol{\Lambda}_{t-1} + \boldsymbol{\beta}_t}{\boldsymbol{\gamma}_t \odot \boldsymbol{\Lambda}_{t-1} + \boldsymbol{\delta}_t}, \quad (16)$$

$$\mathbf{M}_t = \begin{pmatrix} \boldsymbol{\alpha}_t & \boldsymbol{\beta}_t \\ \boldsymbol{\gamma}_t & \boldsymbol{\delta}_t \end{pmatrix} = \begin{pmatrix} 1 + \bar{\mathbf{p}}_t \odot \boldsymbol{\phi}_t & \bar{\mathbf{a}}_t^2 \odot \boldsymbol{\phi}_t \\ \bar{\mathbf{p}}_t & \bar{\mathbf{a}}_t^2 \end{pmatrix}. \quad (17)$$

**Interpretation: precision track as uncertainty-driven gating.** While Equation (16) is nonlinear in  $\lambda_{t-1}$ , it admits a simple “gate-like” rearrangement:

$$\lambda_t = \underbrace{(\bar{a}_t^2 + \bar{p}_t \odot \lambda_{t-1})^{-1} \odot \lambda_{t-1}}_{(\text{confidence history})} + \underbrace{\phi_t = \mathbf{k}_t^2 \odot \Lambda_t^v}_{(\text{confidence in current token})}. \quad (18)$$

The shared denominator  $\bar{a}_t^2 + \bar{p}_t \odot \lambda_{t-1}$  introduces *history dependence*: as accumulated precision grows, the model naturally becomes more selective about incorporating new evidence. We name its inverse  $\rho_t := \mathbf{1} \odot (\bar{a}_t^2 + \bar{p}_t \odot \lambda_{t-1})$ , so the history term of Equation (18) is  $\rho_t \odot \lambda_{t-1}$ . The same factor  $\rho_t$  reappears in the mean recursion below as the forget gate  $\mathbf{f}_t = \rho_t \odot \bar{\mathbf{a}}_t$  (Theorem 2), tying the precision and mean tracks together and inducing the input/forget-gate behaviour of the mean update (Equation (19)).

**Corollary 1.1** (Precision Updates via Parallel Prefix Scan). *Given  $\lambda_0$  and matrices  $\{\mathbf{M}_t\}_{t=1}^T$  from Theorem 1, the posterior precisions  $\{\lambda_t\}_{t=1}^T$  can be computed by a parallel prefix scan over  $\{\mathbf{M}_t\}$  with  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth.*

**Theorem 2** (Mean Update as Affine Transformations). *Let  $\eta_t := \lambda_t \odot \mu_t$  be the posterior information mean and  $\Lambda_t^v$  the value precision. Given the precision path  $\{\lambda_t\}$ , the information mean evolves affinely:*

$$\eta_t = \underbrace{(\bar{a}_t^2 + \bar{p}_t \odot \lambda_{t-1})^{-1} \odot \bar{\mathbf{a}}_t \odot \eta_{t-1}}_{\mathbf{f}_t \text{ (history-dependent forget gate)}} + \underbrace{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}_{\text{token evidence}}. \quad (19)$$

**Corollary 2.1** (Mean Updates via Parallel Prefix Scan). *The posterior information means  $\{\eta_t\}_{t=1}^T$  are computable via parallel prefix scan over the affine transformations in Theorem 2 in  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth.*

**Special cases:** Under deterministic ( $\mathbf{p}_t = 0$ ) and linear time-invariant (LTI) settings, the KLA updates reduce to convolutions computable in  $\mathcal{O}(\log T)$  time via FFT (see Theorem 3 in the Appendix).

### 4.3. KLA as an In-Context Online Learner

Recent sub-quadratic mixers are increasingly unified through an *online-learning* / fast-weight lens, first articulated for SSMs by Longhorn (Liu et al., 2025) and developed in follow-up linear-attention and delta-rule work (Schlag et al., 2021; Yang et al., 2023; 2024; Katharopoulos et al., 2020; Dao & Gu, 2024): each recurrent update is the closed-form minimiser of a per-token objective that trades a *proximal* term (stay close to the decayed memory) against a *fit* term (absorb the current token evidence).

We can show that KLA’s equivalent moment-form recursion admits an online update that is a *gated RNN update* solving

a local *precision-weighted least-squares* objective. Reading the information-mean recursion of Theorem 2 in moment coordinates,  $\mu_t = \eta_t \odot \lambda_t$ , collapses the predict/update pair into this single gated recurrence - the diagonal/scalar instance of the KLA row of Table 3:

**Corollary 2.2** (Posterior-mean recursion as a gated RNN update). *The moment-form posterior mean  $\mu_t = \eta_t \odot \lambda_t$  of the information recursion in Theorem 2 satisfies the gated recurrence*

$$\mu_t = \bar{\mathbf{a}}_t \odot \left(1 - \frac{\mathbf{k}_t^2 \odot \Lambda_t^v}{\lambda_t}\right) \odot \mu_{t-1} + \frac{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}{\lambda_t}, \quad (20)$$

*the diagonal/scalar instance of the KLA state update in Table 3.*

We prove Corollary 2.2 in Section D. Concretely, this gated update is the exact minimiser  $\mu_t = \arg \min_{\mu} L_t$  of the precision-weighted least-squares objective

$$L_t(\mu) = \Lambda_t^v \|\mathbf{v}_t - \mathbf{k}_t \odot \mu\|^2 + \rho_t \odot \lambda_{t-1} \|\mu - \bar{\mathbf{a}}_t \odot \mu_{t-1}\|^2, \quad (21)$$

balancing a fit term (weighted by the observation precision  $\Lambda_t^v$ ) against a proximal term anchoring  $\mu$  to the propagated mean  $\bar{\mathbf{a}}_t \odot \mu_{t-1}$  with weight the prior precision  $\lambda_t^{\text{prior}} = \rho_t \odot \lambda_{t-1}$  - the history-dependent factor  $\rho_t$  of Equation (18), i.e. the prior-to-previous precision ratio. Table 3 places KLA in this template alongside representative delta-rule and gated mixers. The process-noise parameter  $\bar{\mathbf{p}}_t$  never appears explicitly in the state update; it enters only *implicitly* through the matrix-Möbius precision recursion of Theorem 1, via the state-dependent factor  $\rho_t = \mathbf{1} \odot (\bar{a}_t^2 + \bar{p}_t \odot \lambda_{t-1})$ . Setting  $\bar{\mathbf{p}}_t = 0$  collapses  $\rho_t$  to a constant and freezes this gate, replacing the Möbius-structured nonlinear update with a linear one: KLA reduces to a Mamba-2-style fixed-decay recurrence (the design point of concurrent steady-state Kalman attention), i.e. a recursive least-squares variant (von Oswald et al., 2025; Peng et al., 2026).

### 4.4. KLA as a Drop-in Probabilistic Sequence Mixer

In order to instantiate the above parallel Bayesian filter as a neural network layer, we parametrise the observation value and precision ( $\mathbf{v}_t, \Lambda_t^v$ ) as well as the observation and read-out operators ( $\mathbf{k}_t, \mathbf{q}_t$ ) in terms of the layer input sequence  $x_{1:T}$ . The observation variance is parametrised such that it is positive. For the continuous stochastic prior (Section 4.1), we treat  $\mathbf{a}_t$  and  $\mathbf{p}_t$  as learnable, time-invariant parameters  $\mathbf{a}$  and  $\mathbf{p}$ , respectively, in contrast to Mamba where  $\mathbf{a}_t$  is token-dependent and time-varying.

The block architecture follows a fused MLP architecture as shown in Figure 3a. Following the Mamba design, we employ standard scaffolding components paired with the KLA sequence mixer: a 1D causal convolution with kernel size 4 and SiLU activation, along with residual connections

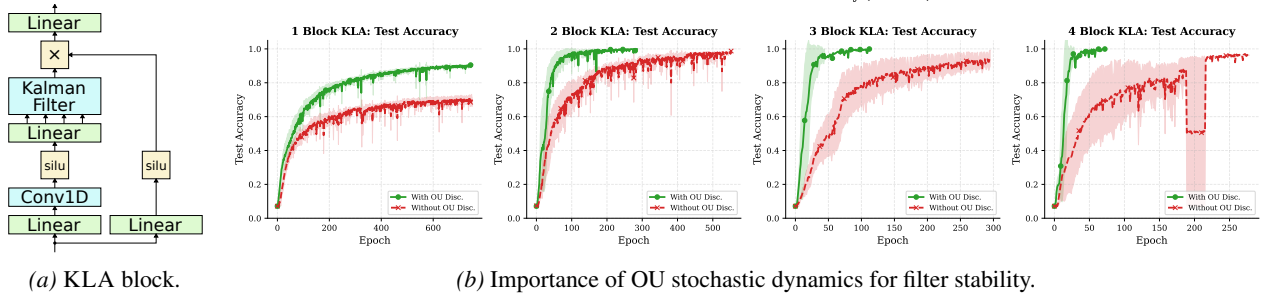


Figure 3. (a) The KLA block follows Mamba’s fused-MLP design, with the Kalman filter as a drop-in mixer (full architecture in Section A). (b) Ablating the OU prior dynamics and discretisation (Section 4.1): OU discretisation improves accuracy and learning stability, especially for deeper models.

| Method                                          | Local online objective $\mathcal{L}_t(\mathbf{S})$                                                                                                                                                                         | State update*                                                                                                                                                                                         | Gates from                                                                              |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <i>Correlation write into a (decayed) state</i> |                                                                                                                                                                                                                            |                                                                                                                                                                                                       |                                                                                         |
| Linear Attn.                                    | $\ \mathbf{S} - \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{v}_t \rangle$                                                                                                                      | $\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$                                                                                                                                    | -                                                                                       |
| Mamba-1 (S6)                                    | $\ \mathbf{S} - \bar{\mathbf{A}}_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{v}_t \rangle$                                                                                                   | $\mathbf{S}_t = \bar{\mathbf{A}}_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$                                                                                                                 | $\mathbf{A}, \bar{\mathbf{A}}_t$                                                        |
| Mamba-2                                         | $\ \mathbf{S} - \alpha_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{v}_t \rangle$                                                                                                             | $\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$                                                                                                                           | $a, \alpha_t$                                                                           |
| <i>Delta (residual) write</i>                   |                                                                                                                                                                                                                            |                                                                                                                                                                                                       |                                                                                         |
| DeltaNet (Schlag et al., 2021)                  | $\ \mathbf{S} - \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \beta_t (\mathbf{v}_t - \mathbf{S}_{t-1}^\top \mathbf{k}_t) \rangle$                                                                       | $\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$                                                                      | $\beta_t$                                                                               |
| Gated DeltaNet                                  | $\ \mathbf{S} - \alpha_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \beta_t (\mathbf{v}_t - (\alpha_t \mathbf{S}_{t-1})^\top \mathbf{k}_t) \rangle$                                                   | $\mathbf{S}_t = \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$                                                             | $\alpha_t, \beta_t$                                                                     |
| KDA (Team et al., 2025)                         | $\ \mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \beta_t (\mathbf{v}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{k}_t) \rangle$                                           | $\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$                                                         | $\mathbf{D}_t, \beta_t$                                                                 |
| GDN-2 (Hatamizadeh et al., 2026)                | $\ \mathbf{S} - \mathbf{D}_t \mathbf{S}_{t-1}\ _F^2 - 2\langle \mathbf{S}^\top \mathbf{k}_t, \mathbf{w}_t \mathbf{v}_t - (\mathbf{D}_t \mathbf{S}_{t-1})^\top \mathbf{b}_t \mathbf{k}_t \rangle$                           | $\mathbf{S}_t = (\mathbf{I} - \mathbf{k}_t (\mathbf{b}_t \mathbf{k}_t)^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \mathbf{v}_t)^\top$                                           | $\mathbf{D}_t, \mathbf{b}_t, \mathbf{w}_t$                                              |
| <i>Bayesian filtering write</i>                 |                                                                                                                                                                                                                            |                                                                                                                                                                                                       |                                                                                         |
| KLA (Ours)                                      | $\underbrace{\Lambda_t^{\text{prior}}}_{\text{uncertainty- (precision-) weighted least squares}} \ \mathbf{S} - \bar{\mathbf{A}}_t \mathbf{S}_{t-1}\ _F^2 + \Lambda_t^v \ \mathbf{S}^\top \mathbf{k}_t - \mathbf{v}_t\ ^2$ | $\mathbf{S}_t = \bar{\mathbf{A}}_t \left( \mathbf{I} - \frac{\mathbf{k}_t^2 (\Lambda_t^v)^\top}{\Lambda_t} \right) \mathbf{S}_{t-1} + \frac{\mathbf{k}_t (\Lambda_t^v \mathbf{v}_t)^\top}{\Lambda_t}$ | $\bar{\mathbf{A}}, \bar{\mathbf{P}}, \Lambda_t^v, \text{Möbius recursion } (\Lambda_t)$ |

Table 3. Local online objectives and state updates across linear-attention mixers. **Orange** marks the forget gate (coefficient of  $\mathbf{S}_{t-1}$ ), **green** the write gate (coefficient of  $\mathbf{k}_t \mathbf{v}_t^\top$ ), and **blue** the quantities unique to KLA - the precision weights in the objective (the only *precision-weighted* least-squares in the table, as marked by the underbrace) and the Möbius recursion supplying  $\Lambda_t$ . Subscript  $t$  denotes per-token learned quantities; symbols without subscript denote global parameters. KLA differs structurally by deriving its gating from a coupled recursion - the matrix-Möbius precision update of Theorem 1 - whose solution  $\Lambda_t$  supplies the precision-ratio gain visible in both forget and write terms.

\* For two matrices  $\mathbf{A}, \mathbf{B}$  of equal shape we overload the fraction  $\mathbf{A}/\mathbf{B}$  to denote *element-wise* (Hadamard) division; products between vectors of equal dimension (e.g.  $\mathbf{b}_t \mathbf{k}_t, \mathbf{w}_t \mathbf{v}_t, \Lambda_t^v \mathbf{v}_t$ ) are element-wise. Vector outer products such as  $\mathbf{k}_t \mathbf{v}_t^\top$  and  $\mathbf{k}_t^2 (\Lambda_t^v)^\top$  form  $d_k \times d_v$  matrices in the standard way.

and simple nonlinearities. See Figure 7 and Appendix A for more architecture and implementation details. We implement efficient Triton (Tillet et al., 2019) kernels for the Linear and Möbius parallel scan primitives; however, we do not fuse the whole block into a single kernel, which has been shown to be necessary in order to minimise memory movement and achieve peak performance in other SSMs, especially for the backward pass. Additionally, the KLA posterior mean recurrence can be unrolled into an equivalent matrix multiplication with a lower-triangular “attention” matrix; we visualise this structure in Figure 10 (Appendix E).

#### 4.5. Training Loss

We use two training losses. (i) **Posterior mean:** logits  $\ell_t = g_\theta(\mathbf{y}_t)$  are read out from the posterior mean  $\mathbf{y}_t = \mathbf{q}_t \odot \boldsymbol{\mu}_t$  and

trained with cross-entropy. (ii) **Log marginal likelihood:** we minimise the negative log marginal likelihood of the output token under the latent posterior (a Monte-Carlo objective), which provides an additional training signal for the posterior variances; the equations and details are given in Appendix A.

## 5. Experiments

We evaluate KLA as a *drop-in sequence mixer* for language modelling. Although KLA is derived from continuous-time Gaussian filtering, our goal is pragmatic: to test whether a Bayesian filtering view of selection/attention can retain the efficiency and accuracy of modern deterministic mixers, while yielding interpretable *uncertainty-driven gating* behaviour, and, crucially, whether such a probabilistic primitive can be *stacked* into deep networks and trained at scale.



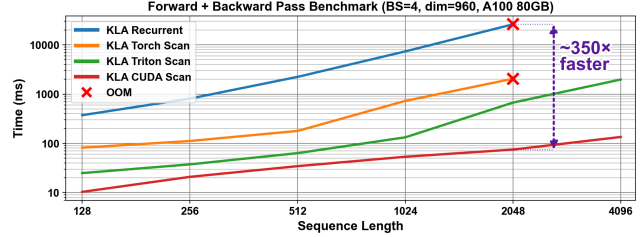
**Experimental questions.** We address four questions:

- **(Q1) Compute and scaling:** how does the parallel implementation of KLA scale with sequence length, and how does it compare to a recurrent (time-stepped) Kalman implementation in wall-clock and memory?
- **(Q2) Model quality as a single-block primitive:** as a drop-in mixer block, does KLA match or improve upon deterministic sub-quadratic alternatives (SSMs / linear attention) on controlled synthetic language-modelling tasks that probe core LM skills (long-range dependence, retrieval, and compression)?
- **(Q3) State-tracking expressivity:** does the fractional-linear structure induced by information-form filtering translate into stronger state-tracking capabilities (e.g., permutation composition) than linear SSM/attention baselines, while retaining scan-parallelisability?
- **(Q4) Stacking and scale:** can KLA be stacked into deep architectures and pretrained at the billion-token scale, matching attention and SSM/GLA backbones as a *standalone* mixer and *complementing* attention as a hybrid on zero-shot commonsense benchmarks?

Finally, we ablate architectural choices within the KLA block to isolate what enables stable deep stacking of a probabilistic primitive.

### 5.1. Baselines

We compare KLA against softmax attention and four recent state-of-the-art sub-quadratic mixers spanning the main architectural paradigms for efficient sequence modelling. **GPT** (Vaswani et al., 2017): a standard softmax-attention transformer, included as a strong  $\mathcal{O}(T^2)$  reference in the pretraining setting. (i) **Recurrent architectures.** **mLSTM** (Beck et al., 2024): a modern LSTM variant featuring multiplicative interactions between hidden states and inputs, exponential gating, and improved training parallelisation. (ii) **Structured state-space models.** **Mamba** (Gu & Dao, 2023): a widely used selective SSM with input-dependent state-transition matrices and gating, discussed in Section 3. **GDN** (Gated DeltaNet) (Yang et al., 2024): a delta-rule-based state-space model with selective gating for memory updates, combining principles from associative memory with modern SSM architectures and representing the current state of the art in SSM-based language modelling. (iii) **Linear attention.** **GLA** (Gated Linear Attention) (Yang et al., 2023): a linear-attention variant that unifies several modern linear RNN/SSM formulations, including Mamba, under a common framework. Throughout, models are matched on parameter count and effective state size; full configurations are in Appendix G.



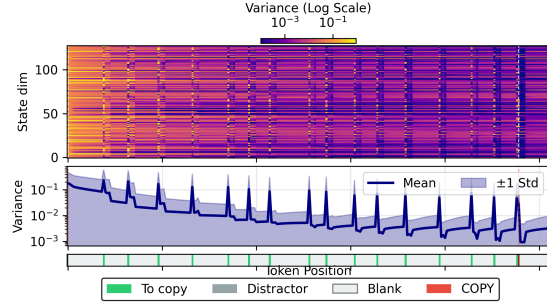
**Figure 4. Compute scaling of parallel KLA.** Forward+backward (training) runtime vs. sequence length across the four KLA implementations: a naive recurrent (time-stepped) Kalman update, a Torch associative scan, a custom Triton associative-scan kernel, and a custom CUDA Möbius-scan kernel that avoids materialising expanded states in HBM. The scan-based variants match the  $\mathcal{O}(\log T)$  profile of modern SSM/GLA mixers; the recurrent baseline OOMs at long sequence lengths. The forward-only (prompt-processing) curve and full setup are in Appendix E.3.

### 5.2. Compute Scaling of Parallel KLA

Scaling probabilistic primitives to long contexts while retaining efficient training parallelism remains comparatively underexplored, and competitive throughput is won not by asymptotics alone but by mathematical reparameterisation and hardware-aware programming acting *in sync*. To make this concrete, we benchmark four KLA implementations that progressively expose this interplay: (i) a naive recurrent (time-stepped) Kalman update; (ii) a mathematically parallel formulation via a Torch associative scan; (iii) a hardware-optimised custom Triton associative-scan kernel; and (iv) a custom CUDA kernel that implements the Möbius scan directly with a fused scan that *avoids materialising the expanded (lifted) states in HBM*, keeping the recurrence in fast on-chip (SRAM) memory - the same kernel-fusion principle that underpins the selective-scan implementation of Mamba (Gu & Dao, 2023). The scan-based implementations scale efficiently with sequence length compared to the recurrent counterpart, confirming that *information-form filtering admits the same scan-parallel  $\mathcal{O}(\log T)$  profile as modern SSM/GLA-style mixers*. Crucially, the Torch scan captures only the reparameterisation, whereas the CUDA kernel adds the memory-traffic savings on top: at  $T=2048$  - the longest sequence the recurrent baseline fits before running out of memory - the CUDA kernel is roughly  $350\times$  faster than the classic recurrent Kalman update, and this gap widens with  $T$  (the recurrent update OOMs at  $T=4096$ ). At pre-training scale such per-step gains compound across billions of tokens and thousands of optimisation steps into hours-to-days of saved GPU wall-clock, and correspondingly more efficient and sustainable use of expensive accelerator resources. The forward+backward (training) runtime curve is shown in Figure 4; the forward-only (prompt-processing) curve and full setup are in Appendix E.3.

| Algorithm          | Compression  | Memorization | Context Recall | Noisy Recall | Fuzzy Recall | Selective Copy |
|--------------------|--------------|--------------|----------------|--------------|--------------|----------------|
| GDN                | 65.53        | <b>99.93</b> | <b>99.94</b>   | <b>99.95</b> | 37.04        | 90.30          |
| GLA                | 49.45        | <b>99.99</b> | 73.60          | 85.24        | 18.22        | 82.41          |
| Mamba              | 78.35        | <b>99.96</b> | <b>99.92</b>   | <b>99.93</b> | 30.83        | 80.60          |
| mLSTM              | 57.17        | <b>99.98</b> | <b>99.98</b>   | <b>99.99</b> | 25.43        | 36.61          |
| <b>KLA (Ours)</b>  | 85.03        | 98.87        | <b>99.95</b>   | <b>99.93</b> | <b>45.70</b> | 90.67          |
| <b>KLA+ (Ours)</b> | <b>88.87</b> | <b>99.94</b> | <b>99.94</b>   | <b>99.95</b> | 43.32        | <b>91.45</b>   |

(a) MAD-Lab accuracy (%).



(b) Posterior variance on Selective Copy.

Figure 5. **KLA as a single-block primitive: quality and uncertainty.** (a) Test accuracy (% ,  $\uparrow$ ) on the MAD synthetic-LM suite for GDN, GLA, Mamba, mLSTM, KLA, and probabilistic decoding KLA +; all methods are matched on parameter count and effective state size, with darker shading = higher accuracy. (b) Posterior variance on Selective Copy: variance decreases as evidence accumulates while sharp spikes align with copy-relevant tokens, an interpretable filtering/selection signal.

### 5.3. KLA as a Single-Block Primitive on Synthetic Language Modelling

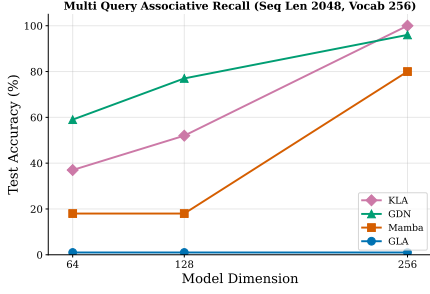
**MAD synthetic LM suite.** We evaluate KLA and the baselines as a single-block primitive on the MAD-Lab benchmark (Poli et al., 2024), a set of six synthetic token-manipulation tasks that test different LM skills (Table 7). We match parameter counts and effective state sizes and use a common encoder-decoder training protocol across all methods (dataset details in Appendix F; hyperparameters in Appendix G). Evaluating as a single block isolates the contribution of the *update mechanism itself*, free of confounds from depth or memory budget.

**Results.** Figure 5a shows that KLA is competitive across the suite and best-in-class precisely where *selection under uncertainty* is required: it attains the top score on *Fuzzy Recall* (45.7) and *Selective Copy* (90.7), the latter a task popularised by Mamba (Gu & Dao, 2023). Notably, whereas Mamba’s selectivity is driven by *token-dependent, time-varying* transition dynamics, KLA keeps its dynamics parameters ( $a$ ,  $\Delta$ ,  $p$ ) as time-invariant global parameters and instead derives selectivity from the *uncertainty ratios* of the Möbius precision recurrence (Equation (16)). On *Compression* - which measures how faithfully a model can reconstruct its context from a single final hidden-state vector - KLA (85.0) and KLA+ (88.9) lead all baselines, consistent with information-form filtering acting as a near least-squares-optimal context compressor. The one mild exception is pure *Memorization* (98.9 vs.  $\sim 99.9$  for the strongest baselines): we read this small gap as KLA favouring *adaptive in-context filtering* over rote storage of a static key-value dictionary into weights, a sensible trade-off given its uncertainty-weighted inductive bias. Finally, probabilistic decoding (KLA+, marginalisation under the learned posterior) further improves *Selective Copy* and *Compression* and closes the *Memorization* gap, indicating that propagating the full predictive distribution - i.e. second-order (variance) statistics - through the loss in a principled

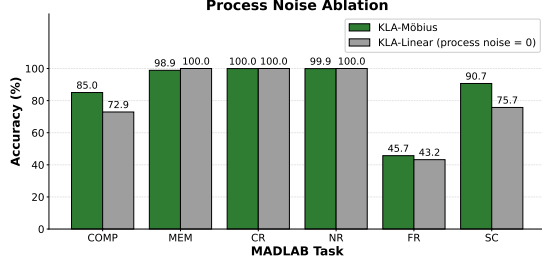
manner yields measurable gains and is a promising avenue for future research.

**Mechanistic behaviour: posterior uncertainty visualisations.** To characterise how KLA filters information, we visualise the posterior variance on Selective Copy (Figure 5b). Rather than claiming calibrated uncertainty estimation, we use it as a diagnostic of the model’s internal filtering dynamics. In Selective Copy, the posterior variance generally decreases over time as evidence accumulates, while sharp variance spikes align with copy-relevant positions, indicating timesteps where the model treats the input as especially informative for updating its belief.

**Long-context associative recall (MQAR).** As a stress test of the single-block primitive under heavy memory load, we evaluate on Multi-Query Associative Recall (MQAR) (Arora et al., 2023), which directly probes the capacity of a fixed-size recurrent state to store and retrieve many key-value associations. Following the Zoology benchmark, we adopt a considerably harder configuration than typically studied - sequence length  $T=2048$  with vocabulary size  $V=256$  - which directly probes the *storage-capacity* limits of fixed-size states. KLA consistently outperforms Mamba across all dimensions and substantially outperforms GLA (which fails to learn the task in this extreme setting), reaching near-perfect accuracy ( $> 95\%$ ) at dimension 256; GDN’s delta rule is stronger at the lowest dimensions, where it also operates with a larger effective state. We attribute KLA’s robustness to its Möbius precision updates (Equation (18)), which adaptively down-weight new observations once the state saturates — an implicit, learned compression mechanism analogous to selective pattern storage in high-capacity associative memories (Storkey & Valabregue, 1997), without requiring explicit dimension scaling. The full dimension sweep is shown in Figure 6a; the data configuration is given in Appendix E.2.



(a) Long-context MQAR accuracy vs. dimension.



(b) Process-noise ablation on MAD tasks.

**Figure 6. Single-block analyses of KLA.** (a) Long-context MQAR ( $T=2048$ ,  $V=256$ ): KLA reaches near-perfect accuracy ( $> 95\%$ ) at dimension 256, outperforming GDN and Mamba, while GLA fails to learn the task in this challenging setting. (b) Process-noise ablation on MAD tasks: fixing  $\mathbf{p}_t=0$  (deterministic dynamics) collapses the Möbius precision recurrence to a fixed-gate linear recurrence, degrading the selection-heavy tasks (Selective Copy, Compression) while leaving memorisation and exact recall unchanged (full per-task breakdown in Table 6, Appendix E.1).

#### 5.4. State Tracking and Permutation-Group Expressivity

**Why  $A_5$  matters.** A central open question is whether sub-quadratic sequence models can *track state* beyond highly-parallel shortcut computation (Merrill & Sabharwal, 2023; Merrill et al., 2024). Recent theory shows that common linear/diagonal SSMs (e.g. S4, Mamba), despite their recurrent form, share the bounded-depth expressivity limits of transformers, they are simulable by highly-parallel circuit classes (informally  $TC^0$ ) and cannot solve inherently sequential state-tracking whose difficulty grows with sequence length. A canonical example is the word problem for the alternating group  $A_5$  (the smallest non-solvable subgroup of  $S_5$ ), which is  $NC^1$ -complete and serves as a minimal benchmark for *hard state tracking* (Merrill et al., 2024).

**Result.** Following Merrill et al. (2024), Figure 1(a) shows that KLA solves the  $A_5$  task at constant depth with only 1-2 layers, whereas linear SSM/attention baselines require depth that grows with sequence length in this regime. Among the parallel-trainable architectures we consider, KLA is the only one to solve the task at constant depth; concurrent work shows linear RNNs can also acquire state tracking through negative eigenvalues (Grazzi et al., 2025), whereas KLA achieves it through the nonlinearity of the Möbius precision recurrence. This is consistent with KLA’s *fractional-linear* (information-form) updates: each step stays scan-parallelisable, yet the state evolution is richer than a linear recurrence, placing KLA beyond  $TC^0$  while preserving the parallel-scan structure of efficient SSM/linear-attention models.

#### 5.5. Stacking KLA for Language Modelling at Scale

The experiments above isolate KLA as a single (1-2 block) primitive. We now ask whether it can be *stacked into deep networks and trained at scale*: we pretrain KLA and all baselines from scratch on FineWeb-Edu at **45M** (1.8B tokens) and

**180M** (10.9B tokens) parameters, with matched optimiser and token budget, reporting zero-shot accuracy on eight commonsense benchmarks (full protocol in Appendix G). To our knowledge this is the first large-scale from-scratch pretraining of a stacked probabilistic-filtering primitive propagating belief states between blocks, and we regard stable training at this scale as a contribution in itself. Stacking also raises an interface question absent for deterministic mixers: KLA emits a full posterior at every step; as a first iteration we propagate only the *mean* between blocks (a single posterior sample performs comparably), leaving full-posterior propagation to future work.

**Standalone KLA is competitive.** Table 4 shows KLA is competitive with softmax attention, Mamba, and GDN at both scales (with per-task strengths on HellaSwag and PIQA). Its advantage is largest at shallow depth and narrows as baselines add layers (in our 24-layer experiments), as predicted by our expressivity analysis (Section 5.4). It underperforms on the *clean* copying task LAMBADA, consistent with uncertainty-weighted updates favouring robust over verbatim recall (cf. Section 5.3).

**A single KLA layer improves a GPT.** Unlike prior hybrids that interleave many layers, we replace only the *final* layer of a softmax-attention GPT with an SSM-like block (KLA, Mamba, or GDN), motivated by evidence that uncertainty concentrates in later layers (Agarwal et al., 2025). At 180M, GPT+KLA improves over pure GPT and over the analogous GPT+Mamba and GPT+GDN hybrids on average and most tasks (Table 4; Figure 1(b)), showing KLA is an effective drop-in *complement* to attention, not merely a substitute.

| Model               | LAMB.<br>acc ↑ | HellaS.<br>acc <sub>n</sub> ↑ | PIQA<br>acc ↑ | Arc-E<br>acc ↑ | Arc-C<br>acc <sub>n</sub> ↑ | WinoGr.<br>acc ↑ | OBQA<br>acc ↑ | BoolQ <sup>†</sup><br>acc ↑ | Average<br>acc ↑ |
|---------------------|----------------|-------------------------------|---------------|----------------|-----------------------------|------------------|---------------|-----------------------------|------------------|
| GPT-45M             | <b>16.00</b>   | 28.60                         | 58.00         | 40.20          | 20.60                       | 50.00            | <b>27.40</b>  | 37.40                       | 34.78            |
| Mamba-45M           | <u>9.20</u>    | <b>29.20</b>                  | <b>59.80</b>  | 37.20          | <b>25.40</b>                | 48.80            | 25.40         | 61.60                       | 37.08            |
| GDN-45M             | <b>16.00</b>   | 28.20                         | 58.40         | 39.00          | <u>23.40</u>                | <b>52.60</b>     | <u>27.20</u>  | 45.20                       | 36.25            |
| KLA-45M             | 7.60           | <u>28.80</u>                  | <u>58.60</u>  | <u>40.60</u>   | 23.20                       | <u>52.20</u>     | <b>27.40</b>  | <b>63.00</b>                | <u>37.68</u>     |
| <b>GPT+KLA-45M</b>  | <b>16.00</b>   | 28.20                         | <u>58.60</u>  | <b>42.80</b>   | 23.00                       | 50.80            | 24.40         | <u>62.80</u>                | <b>38.33</b>     |
| GPT-180M            | <b>28.80</b>   | 32.20                         | <u>63.40</u>  | 51.00          | 27.00                       | 49.20            | 31.20         | 54.80                       | 42.20            |
| Mamba-180M          | 24.00          | 30.60                         | 63.00         | 51.00          | 27.20                       | <b>53.80</b>     | <b>33.00</b>  | 55.80                       | 42.30            |
| GDN-180M            | 27.80          | 31.60                         | 60.40         | 45.60          | 25.80                       | 53.40            | 30.00         | <u>60.80</u>                | 41.93            |
| KLA-180M            | 22.40          | <u>34.00</u>                  | 63.00         | 50.00          | 26.00                       | <u>53.60</u>     | <u>32.40</u>  | <u>60.80</u>                | 42.78            |
| <b>GPT+KLA-180M</b> | <b>30.20</b>   | <b>34.40</b>                  | <b>64.20</b>  | <b>55.80</b>   | <b>29.00</b>                | 51.80            | 31.00         | <b>63.20</b>                | <b>44.95</b>     |

Table 4. **Language modelling at two academic scales.** Zero-shot accuracy (%) on eight commonsense benchmarks at 45M (1.8B tokens) and 180M (10.9B tokens). **Bold** = best within each size block, underline = second. Rows above the dashed line are standalone mixers; the row below it is the hybrid GPT+KLA, in which only the *final* attention layer of GPT is replaced with a KLA block. Per-size comparisons against the analogous GPT+Mamba and GPT+GDN hybrids are in Figure 1(b) and Section E. <sup>†</sup>BoolQ has a  $\sim 62\%$  majority-class baseline; scores near this value should be read with caution and we draw no conclusions from the BoolQ column in isolation.

## 5.6. Ablations

Finally, we isolate the two modelling choices that matter most. Stable *deep stacking* of a probabilistic primitive was possible *only* because of the mean-reverting Ornstein-Uhlenbeck (OU) prior (Section 4.1, Figure 2): naive stacking was unstable and standard fixes (gradient clipping, lower learning rates) did not help at the root, whereas the OU prior keeps the latent dynamics bounded, letting KLA stack to 12 and 24 blocks stably with no gradient explosions (gradient clipping 3.0 for all models; Figure 3b confirms improved stability and accuracy on Selective Copy, especially at depth) - to our knowledge the first stable backpropagation through stacked blocks that propagate belief states between layers and through time. A second ablation (Figure 6b, Appendix E.1) isolates the process-noise parameter  $\mathbf{p}_t$  as the source of KLA’s nonlinear gating:  $\mathbf{p}_t$  enters only through the Möbius precision recurrence (Theorem 1), via the state-dependent factor  $\rho_t = \mathbf{1} \oslash (\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1})$ , so setting  $\mathbf{p}_t=0$  freezes  $\rho_t$  to a constant and collapses the recurrence to a fixed-gate linear update. This costs both expressivity (the gate no longer adapts to accumulated history) and stability: without injected process noise the posterior precision  $\lambda_t$  accumulates unbounded, making the filter overconfident, whereas a non-zero  $\mathbf{p}_t$  caps it and maintains a fading memory. Removing process noise leaves memorisation and exact recall essentially unchanged but degrades the selection-heavy tasks (Selective Copy  $-14.9$ , Compression  $-12.1$  points), a 4.7-point average drop (Table 6).

## 6. Conclusion

In this paper, we propose a scalable probabilistic sequence-modelling primitive, the KLA layer, which treats language modelling as a state estimation problem from noisy

observations. We show that posterior inference in these models is tractable for very long sequences via parallel scans, and that the resulting gating mechanisms are strictly more expressive than those of modern deterministic linear SSMS and RNNs. Our experiments indicate the promise of KLA as a scalable primitive within the language and probabilistic sequence-modelling toolbox.

## 7. Limitations and Future Work

This paper is a *first iteration* on a fundamental algorithmic family - exact Bayesian filters as parallelisable sequence-modelling primitives - and we deliberately start from the simplest possible assumptions: diagonal, linear time-invariant (LTI) dynamics with a linear-Gaussian observation model. Accordingly, the focus is to isolate what the Bayesian filtering update contributes in a minimal but impactful setting; most experiments use shallow configurations rather than deep, large-scale pretraining, and we do not claim state-of-the-art performance on web-scale corpora, very long-context regimes, or open-ended generative benchmarks.

This minimal starting point yields a clear capability profile. KLA is already directly usable where its strengths are structural: state tracking and permutation composition, associative recall (MQAR), and, as a single drop-in block, complementing attention in a hybrid with gains that carry over to zero-shot commonsense reasoning. Its current headroom lies in standalone long-context retrieval and raw next-token quality, which we attribute to the deliberately minimal, time-invariant dynamics of this first iteration; relaxing these assumptions is a natural direction for future members of the family.

We have also not explicitly exploited the posterior covariance for applications where uncertainty quantification may



be critical (e.g., epistemic prompt uncertainty, hallucination detection, and out-of-distribution prompt detection), which are likely areas where KLA could excel; we leave this for future work. Finally, because the underlying state-space formulation is inherently continuous, a promising direction is to move beyond the language domain to vision and multi-modal foundation models over continuous modalities, where an explicit belief-state uncertainty may be especially natural.

## Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we believe must be specifically highlighted here. The KLA layer provides principled uncertainty quantification, which could benefit applications requiring calibrated predictions, though, as with any language modelling primitive, deployment should consider potential misuse.

## Acknowledgements

We thank Henry Gouk, Nikolay Malkin, Thomas Lee, and Stephan Kostov for their valuable feedback on this draft at various stages. We also thank Wouter Boomsma, Frederikke Isa Marin, Jose Cano Reyes, and Jude Harris for the informative discussions over the course of this research. This project has received funding from the European Union’s Horizon Europe research and innovation programme under grant agreement No. 101120726. This work was funded by UK Research and Innovation (UKRI) under the UK government’s Horizon Europe funding Guarantee 10085198. We also gratefully acknowledge the support of the UK Engineering and Physical Sciences Research Council (EPSRC), under grant EP/W002876/1 and through a PhD Studentship within the CDT in Machine Learning Systems, School of Informatics, University of Edinburgh (EP/Y03516X/1). The authors acknowledge the use of resources provided by the Isambard-AI National AI Research Resource (AIRR) (McIntosh-Smith et al., 2024). Isambard-AI is operated by the University of Bristol and is funded by the UK Government’s Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

## References

Agarwal, N., Dalal, S. R., and Misra, V. The bayesian geometry of transformer attention. *arXiv preprint arXiv:2512.22471*, 2025.

Alonso, C. A., Sieber, J., and Zeilinger, M. N. State space models as foundation models: A control theoretic overview. In *2025 American Control Conference (ACC)*,

pp. 146–153. IEEE, 2025.

Anderson, B. D. and Moore, J. B. *Optimal filtering*. Courier Corporation, 2005.

Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., and Ré, C. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.

Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xLSTM: Extended long short-term memory. *Advances in Neural Information Processing Systems*, 37: 107547–107603, 2024.

Becker, P., Pandya, H., Gebhardt, G., Zhao, C., Taylor, C. J., and Neumann, G. Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 544–552. PMLR, 2019.

Becker, P., Freymuth, N., and Neumann, G. KalMamba: Towards efficient probabilistic state space models for RL under uncertainty. In *ICML 2024 Workshop: Aligning Reinforcement Learning Experimentalists and Theorists*, 2024.

Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.

Blelloch, G. E. Prefix sums and their applications. In *Synthesis of Parallel Algorithms*, pp. 35–60. Morgan Kaufmann, 1990.

Broadbent, D. E. A mechanical model for human attention and immediate memory. *Psychological review*, 64(3):205, 1957.

Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 conference of the north American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, pp. 2924–2936, 2019.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Taffjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Dao, T. and Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

- Deutsch, J. A. and Deutsch, D. Attention: Some theoretical considerations. *Psychological review*, 70(1):80, 1963.
- Feldman, H. and Friston, K. J. Attention, uncertainty, and free-energy. *Frontiers in human neuroscience*, 4:215, 2010.
- Fraccaro, M., Sønderby, S. K., Paquet, U., and Winther, O. Sequential neural models with stochastic layers. In *Advances in Neural Information Processing Systems*, volume 29, pp. 2207–2215, 2016.
- Grazzi, R., Siems, J., Franke, J. K. H., Zela, A., Hutter, F., and Pontil, M. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations (ICLR)*, 2025.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Gu, A., Goel, K., and Re, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2021.
- Gupta, A., Gu, A., and Berant, J. Diagonal state spaces are as effective as structured state spaces. In *Advances in Neural Information Processing Systems*, volume 35, pp. 22982–22994, 2022.
- Haarnoja, T., Tang, H., Abbeel, P., and Levine, S. Reinforcement learning with deep energy-based policies. In *International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1352–1361. PMLR, 2017.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2555–2565. PMLR, 2019.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- Hatamizadeh, A., Choi, Y., and Kautz, J. Gated deltanet-2: Decoupling erase and write in linear attention. *arXiv preprint arXiv:2605.22791*, 2026.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2022.
- Kalman, R. E. A new approach to linear filtering and prediction problems. *Transactions of the ASME—Journal of Basic Engineering*, 82(Series D):35–45, 1960.
- Karl, M., Sölch, M., Bayer, J., and van der Smagt, P. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. In *Proceedings of the International Conference on Learning Representations*, 2017.
- Karpathy, A. nanochat: The best chatgpt that \$100 can buy, 2025. URL <https://github.com/karpathy/nanochat>.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pp. 5156–5165. PMLR, 2020.
- Khan, M. E. Matrix inversion lemma and information filter. *Honeywell Techonology Solutions Lab, Bangalore, India*, 2005.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Krishnan, R. G., Shalit, U., and Sontag, D. Deep Kalman filters. *arXiv preprint arXiv:1511.05121*, 2015.
- Krishnan, R. G., Shalit, U., and Sontag, D. Structured inference networks for nonlinear state space models. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pp. 2101–2109, 2017.
- Liu, B., Wang, R., Wu, L., Feng, Y., Stone, P., and Liu, Q. Longhorn: State space models are amortized online learners. In *International Conference on Learning Representations*, volume 2025, pp. 95419–95434, 2025.
- Luis, C. E., Bottero, A. G., Vinogradskaya, J., Berkenkamp, F., and Peters, J. Uncertainty representations in state-space layers for deep reinforcement learning under partial observability. *arXiv preprint arXiv:2409.16824*, 2024.
- McIntosh-Smith, S., Alam, S., and Woods, C. Isambard-AI: a leadership-class supercomputer optimised specifically for Artificial Intelligence. In *Proceedings of the Cray User Group*, pp. 44–54. 2024.
- Merrill, W. and Sabharwal, A. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.

- Merrill, W., Petty, J., and Sabharwal, A. The illusion of state in state-space models. *arXiv preprint arXiv:2404.08819*, 2024.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 2381–2391, 2018.
- Muca Cirone, N., Orvieto, A., Walker, B., Salvi, C., and Lyons, T. Theoretical foundations of deep selective state-space models. *Advances in Neural Information Processing Systems*, 37:127226–127272, 2024.
- Nielsen, F. and Garcia, V. Statistical exponential families: A digest with flash cards. *arXiv preprint arXiv:0911.4863*, 2009.
- Paperno, D., Kruszewski, G., Lazaridou, A., Pham, N.-Q., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 1525–1534, 2016.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- Penedo, G., Kydlíček, H., allal, L. B., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., and Wolf, T. The fineweb datasets: Decanting the web for the finest text data at scale. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 30811–30849, 2024.
- Peng, L., Chattopadhyay, A., Zancato, L., Nunez, E., Xia, W., and Soatto, S. Gated kalmanet: A fading memory layer through test-time ridge regression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20212–20222, 2026.
- Poli, M., Thomas, A. W., Nguyen, E., Ponnusamy, P., Deiseroth, B., Kersting, K., Suzuki, T., Hie, B., Ermon, S., Re, C., Zhang, C., and Massaroli, S. Mechanistic design and scaling of hybrid architectures. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 40908–40950. PMLR, 2024.
- Rao, R. P. and Ballard, D. H. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature neuroscience*, 2(1):79–87, 1999.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Särkkä, S. and García-Fernández, Á. F. Temporal parallelization of bayesian smoothers. *IEEE Transactions on Automatic Control*, 66(1):299–306, 2020.
- Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In *International conference on machine learning*, pp. 9355–9366. PMLR, 2021.
- Shaj, V., Becker, P., Büchler, D., Pandya, H., van Duijckeren, N., Taylor, C. J., Hanheide, M., and Neumann, G. Action-conditional recurrent kalman networks for forward and inverse dynamics learning. In *Conference on Robot Learning*, pp. 765–781. PMLR, 2021.
- Shaj Kumar, V., Gholam Zadeh, S., Demir, O., Douat, L., and Neumann, G. Multi time scale world models. *Advances in Neural Information Processing Systems*, 36:26764–26775, 2023.
- Smith, J. T., Warrington, A., and Linderman, S. Simplified state space layers for sequence modeling. In *The Eleventh International Conference on Learning Representations*, 2022.
- Storkey, A. and Valabregue, R. Hopfield learning rule with high capacity storage of time-correlated patterns. *Electronics Letters*, 33:1803 – 1804, 11 1997.
- Team, K., Zhang, Y., Lin, Z., Yao, X., Hu, J., Meng, F., Liu, C., Men, X., Yang, S., Li, Z., et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.
- Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.
- Treisman, A. M. Strategies and models of selective attention. *Psychological review*, 76(3):282, 1969.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

- von Oswald, J., Scherrer, N., Kobayashi, S., Versari, L., Yang, S., Schlegel, M., Maile, K., Schimpf, Y., Sieberling, O., Meulemans, A., et al. Mesanet: Sequence modeling by locally optimal test-time training. *arXiv preprint arXiv:2506.05233*, 2025.
- Wang, K. A., Shi, J., and Fox, E. B. Test-time regression: a unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025.
- Watter, M., Springenberg, J. T., Boedecker, J., and Riedmiller, M. A. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, volume 28, pp. 2746–2754, 2015.
- Yang, G., Hu, E., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., Ryder, N., Pachocki, J., Chen, W., and Gao, J. Tuning large neural networks via zero-shot hyperparameter transfer. *Advances in Neural Information Processing Systems*, 34:17084–17097, 2021.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- Yu, A. and Erichson, N. B. Block-biased mamba for long-range sequence processing. *arXiv preprint arXiv:2505.09022*, 2025.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 4791–4800, 2019.



## Appendix

### Appendix outline

- [Appendix A: Architecture Details \(p. 18\)](#)
  - [Figure 5: Model Architecture Diagram \(p. 18\)](#)
  - [Algorithm 1: Kalman Linear Attention \(p. 19\)](#)
  - [Training Loss](#)
- [Appendix B: Notation \(p. 20\)](#)
- [Appendix C: Extended Background \(p. 20\)](#)
  - [Mobius / Fractional-Linear Transformations \(p. 20\)](#)
  - [Information Form of Gaussian Distributions \(p. 20\)](#)
  - [Kalman and Information Filters \(p. 21\)](#)
- [Appendix D: Theorems and Proofs \(p. 23\)](#)
- [Appendix E: Additional Empirical Results \(p. 27\)](#)
  - [Ablation On The Importance Of Process Noise Parameter \(p. 27\)](#)
  - [Forward Pass Runtime Scaling \(p. 28\)](#)
  - [Equivalent Attention Matrix \(p. 28\)](#)
  - [Attention Map Visualisation \(p. 28\)](#)
- [Appendix F: Datasets and Benchmarks Used \(p. 31\)](#)
  - [MAD LM Suite \(p. 31\)](#)
  - [Long-Context MQAR \(p. 32\)](#)
- [Appendix G: Hyperparameters Used \(p. 33\)](#)
  - [Experimental Protocol \(p. 33\)](#)
  - [Training Hyperparameters \(p. 33\)](#)
  - [MAD-Lab Hyperparameters \(p. 33\)](#)
  - [MQAR Hyperparameters \(p. 33\)](#)
  - [A5 State Tracking Hyperparameters \(p. 34\)](#)



**Algorithm 1** Kalman Linear Attention

---

```

1: Input:  $\mathbf{X} : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D})$ 
2: Output  $\mathbf{Y}^\mu : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D})$ 
3: Output (Optional)  $\mathbf{Y}^\Sigma : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D})$ 
4:  $\mathbf{A}, \mathbf{P}, \Delta : (\mathbf{N}, \mathbf{D}) \leftarrow \text{Parameters}$  ▷ A: Gating/decay, P: Process noise, Δ: Discretisation step size
5:  $\mathbf{K} : (\mathbf{B}, \mathbf{T}, \mathbf{N}, 1) \leftarrow f_{\theta_k}(\mathbf{X})$  ▷ Observation operator
6:  $\mathbf{Q} : (\mathbf{B}, \mathbf{T}, 1, \mathbf{N}) \leftarrow f_{\theta_q}(\mathbf{X})$  ▷ Readout operator
7:  $\mathbf{V}, \Lambda^\vee : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D}) \leftarrow f_{\theta_v, \lambda^\vee}(\mathbf{X})$  ▷ Observation mean and precision,  $\lambda^\vee \in \mathbb{R}^+$ 
8:  $\bar{\mathbf{A}} : (\mathbf{N}, \mathbf{D}) \leftarrow \text{OU-Discretisation}(\mathbf{A}, \Delta)$  ▷ See Equation (8)
9:  $\bar{\mathbf{P}} : (\mathbf{N}, \mathbf{D}) \leftarrow \text{OU-Discretisation}(\mathbf{A}, \mathbf{P}, \Delta)$ 
10:  $\Lambda : (\mathbf{B}, \mathbf{T}, \mathbf{N}, \mathbf{D}) \leftarrow \text{Mobius-Scan}(\mathbf{K}, \Lambda^\vee, \bar{\mathbf{P}}, \bar{\mathbf{A}})$  ▷ matrix form: Equation (22)
11:  $\mathbf{H} : (\mathbf{B}, \mathbf{T}, \mathbf{N}, \mathbf{D}) \leftarrow \text{Linear-Scan}(\mathbf{K}, \Lambda^\vee, \mathbf{V}, \Lambda, \bar{\mathbf{P}}, \bar{\mathbf{A}})$  ▷ information mean  $\boldsymbol{\eta}$ ; matrix form: Equation (23)
12:  $\mathbf{Y}^\mu : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D}) \leftarrow \mathbf{Q}(\mathbf{H}\Lambda^{-1})$ 
13: if Decode Variance then
14:    $\mathbf{Y}^\Sigma : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D}) \leftarrow \mathbf{Q}^2\Lambda^{-1}$ 
15:   Return:  $\mathbf{Y}^\mu, \mathbf{Y}^\Sigma$ 
16: else
17:   Return:  $\mathbf{Y}^\mu$ 
18: end if
    
```

---

**Training loss.** We consider two training losses. Throughout, the decoder is applied to the query-conditioned readout  $\mathbf{y}_t = \mathbf{q}_t \odot \mathbf{z}_t$ ; when the readout noise is zero this is a deterministic linear map, so conditioning on  $\mathbf{z}_t$  or  $\mathbf{y}_t$  is equivalent up to this transformation.

(i) **Posterior mean:** logits  $\ell_t = g_\theta(\mathbf{y}_t)$  are obtained from the posterior-mean readout  $\mathbf{y}_t = \mathbf{q}_t \odot \boldsymbol{\mu}_t$  and trained with cross-entropy.

(ii) **Log marginal likelihood:** minimises the negative log marginal likelihood of the output token  $o_t$  under the latent posterior,

$$\begin{aligned}
 -\log p_\theta(o_t | \mathbf{X}) &= -\log \int \underbrace{p_\theta(o_t | \mathbf{y}_t)}_{\text{decoder}} \underbrace{p_\theta(\mathbf{z}_t | \mathbf{X})}_{\text{KLA posterior}} d\mathbf{z}_t \\
 &\approx -\log \left( \frac{1}{S} \sum_{s=1}^S p_\theta(o_t | \mathbf{y}_t^{(s)}) \right),
 \end{aligned} \tag{24}$$

with  $\mathbf{z}_t^{(s)} \sim p_\theta(\mathbf{z}_t | \mathbf{X})$  and  $\mathbf{y}_t^{(s)} = \mathbf{q}_t \odot \mathbf{z}_t^{(s)}$ . Equivalently, the estimator can be written as a logsumexp:

$$\begin{aligned}
 &-\log p_\theta(o_t | \mathbf{X}) \\
 &\approx -\log \left( \frac{1}{S} \sum_{s=1}^S p_\theta(o_t | \mathbf{y}_t^{(s)}) \right) \\
 &= -\log \sum_{s=1}^S \exp(\log p_\theta(o_t | \mathbf{y}_t^{(s)})) + \log S,
 \end{aligned} \tag{25}$$

which reduces to cross-entropy when  $S=1$ . We show this for a single time-step; it can be trivially extended across timesteps.

## B. Notation

Throughout the paper we use the following convention:

| Symbol                                                                               | Meaning                                                                                          |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $t$                                                                                  | Time index                                                                                       |
| $z_t$                                                                                | Latent state at time $t$                                                                         |
| $v_t$                                                                                | Value (observed token/features) at time $t$                                                      |
| $A_t$                                                                                | State transition matrix                                                                          |
| $P_t$                                                                                | Process noise covariance (discretized: $\bar{P}_t$ )                                             |
| $C_t$                                                                                | Observation (likelihood) matrix                                                                  |
| $\Lambda_t^v$                                                                        | Value precision (inverse observation variance)                                                   |
| $\mathcal{N}(\mu, \Sigma)$                                                           | Gaussian in <i>moment</i> form                                                                   |
| $\mathcal{N}(\eta, \Lambda)$                                                         | Gaussian in <i>canonical</i> form; $\eta = \Lambda\mu$ , $\Lambda = \Sigma^{-1}$                 |
| $\mu_t^{\text{prior}}, \Sigma_t^{\text{prior}}$                                      | Predictive prior mean/cov at $t$ (canonical: $\eta_t^{\text{prior}}, \Lambda_t^{\text{prior}}$ ) |
| $\mu_t, \Sigma_t$                                                                    | Posterior mean/cov at $t$ (canonical: $\eta_t, \Lambda_t$ )                                      |
| <b>Beliefs (linear–Gaussian case):</b>                                               |                                                                                                  |
| $z_t \mid v_{1:t-1} \sim \mathcal{N}(\mu_t^{\text{prior}}, \Sigma_t^{\text{prior}})$ | (predictive prior)                                                                               |
| $z_t \mid v_{1:t} \sim \mathcal{N}(\mu_t, \Sigma_t)$                                 | (posterior)                                                                                      |

**Diagonal (per-dimension) specialisation.** When matrices are diagonal, we identify them with their diagonal vectors and switch to bold lowercase:  $A_t \equiv \mathbf{a}_t$ ,  $B_t \equiv \mathbf{b}_t$ ,  $K_t \equiv \mathbf{k}_t$  (key),  $P_t \equiv \mathbf{p}_t$  (process noise); value precision  $\Lambda_t^v$  is treated as a scalar. All algebra becomes *elementwise*: products use the Hadamard operator  $\odot$ , superscripts like  $\mathbf{a}_t^2$  denote elementwise squares, and  $(\cdot)^{-1}$  on lowercase vectors means elementwise inverse.

| Full (matrix) | Diagonal notation (this paper) | Implication (elementwise)                                                               |
|---------------|--------------------------------|-----------------------------------------------------------------------------------------|
| $A_t$         | $\mathbf{a}_t$                 | $(A_t \mathbf{x}) = \mathbf{a}_t \odot \mathbf{x}$ ; $A_t^2 \Rightarrow \mathbf{a}_t^2$ |
| $K_t$         | $\mathbf{k}_t$                 | $K_t \Sigma K_t^\top \Rightarrow \mathbf{k}_t^2 \odot \Sigma$ (key)                     |
| $P_t$         | $\mathbf{p}_t$                 | Process noise cov.; $\bar{P}_t \Rightarrow \bar{\mathbf{p}}_t$ (discretized)            |
| $\Lambda_t^v$ | $\Lambda_t^v$                  | Scalar value precision (variance = $(\Lambda_t^v)^{-1}$ )                               |

## C. Extended Background

### C.1. Möbius / Fractional-Linear Transformations.

A Möbius (a.k.a. fractional-linear) transformation is the map

$$z \mapsto \frac{az + b}{cz + d}, \quad ad - bc \neq 0.$$

It generalises the familiar affine form ( $c = 0$  gives  $az + b$ ) by introducing an additional denominator term. Each transform is represented (up to multiplication by a nonzero scalar) by a  $2 \times 2$  matrix  $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$  and acts on a scalar  $z$  via the linear–fractional rule

$$M(z) := \frac{az + b}{cz + d}.$$

Composing multiple transforms amounts to multiplying their representing matrices, so associativity and invertibility (whenever  $\det M \neq 0$ ) follow directly. Geometrically, Möbius transforms can be viewed as compositions of translations, rotations/scalings, and a crucial nonlinear ingredient—*inversion*—which affine transformations cannot capture. The denominator also provides a built-in *self-normalisation*: as the state grows, it is automatically rescaled, producing saturation and stability effects reminiscent of gating or normalisation layers, but achieved directly through the update itself.

### C.2. Information Form of Gaussian Distributions and Information Filters.

Gaussians belong to the exponential family (Nielsen & Garcia, 2009), so the posterior can be written in *canonical* (information) form with precision  $\Lambda_t = \Sigma_t^{-1}$  and natural parameter  $\eta_t = \Lambda_t \mu_t$ . Filtering recursions in this parameterisation



are known as *information filters (IF)* (Anderson & Moore, 2005; Khan, 2005). They are algebraically equivalent to the standard (moment) Kalman updates; only the representation of the Gaussian belief differs. Crucially, the canonical posterior updates are *linear-additive* in  $(\eta, \Lambda)$ , a property we leverage for scalable sequence modelling in Section 4.

### C.3. Control vs Observation View Of Token Processing

State-space models can be understood from two complementary perspectives (Figure 8):

**Observation view (traditional SSM/Kalman filter).** In classical Bayesian filtering, input tokens are treated as *noisy observations* of a hidden state. The model assumes: (i) a latent state  $z_t$  evolves over time via learned dynamics, and (ii) each token  $v_t$  is a noisy measurement generated from this state. The goal is to *infer* the hidden state from the observed tokens—asking “what underlying state could have produced these observations?”

**Control view (Mamba/S6).** In contrast, Mamba treats input tokens as *control signals* that directly drive the state. There is no observation noise or inference step; tokens deterministically update the hidden state via  $z_t = A_t z_{t-1} + B_t u_t$ . The state simply accumulates information from inputs—asking “how should I update my memory given this new input?”

**Why this matters.** The observation view naturally provides uncertainty quantification: since tokens are noisy measurements, the model maintains a posterior *distribution* over states rather than a point estimate. This is the foundation of KLA—we treat language modelling as Bayesian filtering where tokens inform our belief about an underlying semantic state.

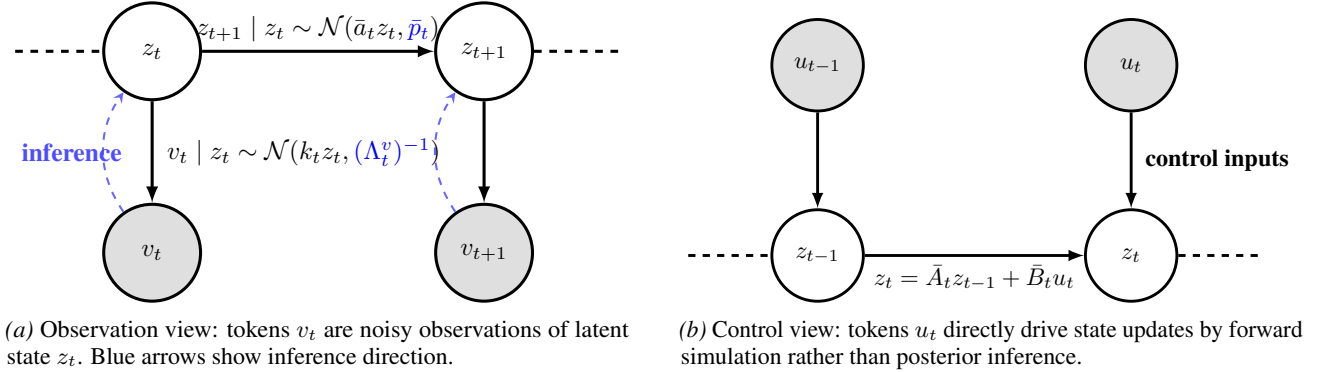


Figure 8. Two views of state-space sequence models. **Left:** Bayesian/observation view - tokens are measurements, inference recovers hidden state. **Right:** Control view (Mamba) - tokens are inputs that deterministically update state.

### C.4. Kalman and Information Filters

#### C.4.1. BAYESIAN FILTERING AS POSTERIOR INFERENCE.

Bayesian filters (Kalman, information-filter variants) are *inference schemes*—procedures that compute posterior beliefs about latent states from noisy data. The Kalman filter assumes a Gaussian linear state-space model: the latent evolves via a (possibly time-varying) linear transition  $A_t$  with unmodelled variability captured by process noise  $Q_t$ , and each observed token feature  $o_t$  is generated by an observation model  $C_t$  with observation noise covariance  $\Sigma_t^{\text{obs}}$ . The filter then computes  $p(z_t | o_{1:t})$  sequentially by combining the dynamics prior with token evidence.

In this linear-Gaussian setting, inference has closed-form solutions: in the *moment parametrisation*  $(\mu_t, \Sigma_t)$  the scheme is called the **Kalman filter**, while in the *canonical parametrisation*  $(\eta_t, \Lambda_t)$  it is called the **information filter**. Both are algebraically equivalent, differing only in how Gaussian beliefs are represented.

**Intuition on noise.** In our linear-Gaussian state-space model, the “noise” terms  $w_t$  and  $v_t$  are not claims that language or the world is intrinsically random; they compactly represent *uncertainty* and unmodeled effects (domain shift, annotation/embedding error, missing context). Their covariances  $P_t$  and  $\Sigma_t^{\text{obs}}$  calibrate trust in dynamics vs. data—larger  $P_t$  yields more adaptivity (faster forgetting), while larger  $\Sigma_t^{\text{obs}}$  leans on the prior—producing principled “gates” via precision ratios. In contrast, ‘noise’ in diffusion models is an *algorithmic corruption* deliberately added in a forward process to define a denoising/score-matching objective; its schedule  $(\beta_t)$  is chosen for optimisation/SNR shaping rather than for model

| Belief / Parameters                                                                                               | Kalman Filter (moment form)                                                                                                                                                                                                              | Information Filter (canonical form)                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prior / Prediction</b><br>$z_t \mid v_{1:t-1} \sim \mathcal{N}(\mu_t^{\text{prior}}, \Sigma_t^{\text{prior}})$ | $\mu_t^{\text{prior}} = A_t \mu_{t-1}$<br>$\Sigma_t^{\text{prior}} = A_t \Sigma_{t-1} A_t^\top + P_t$                                                                                                                                    | $\eta_t^{\text{prior}} = \Lambda_t^{\text{prior}} A_t \Lambda_{t-1}^{-1} \eta_{t-1}$<br>$\Lambda_t^{\text{prior}} = (A_t \Lambda_{t-1}^{-1} A_t^\top + P_t)^{-1}$ |
| <b>Posterior / Update</b><br>$z_t \mid v_{1:t} \sim \mathcal{N}(\mu_t, \Sigma_t)$                                 | $\mu_t = \mu_t^{\text{prior}} + K_t (v_t - C_t \mu_t^{\text{prior}})$<br>$\Sigma_t = (I - K_t C_t) \Sigma_t^{\text{prior}}$<br>$K_t = \Sigma_t^{\text{prior}} C_t^\top (C_t \Sigma_t^{\text{prior}} C_t^\top + (\Lambda_t^v)^{-1})^{-1}$ | $\Lambda_t = \Lambda_t^{\text{prior}} + C_t^\top \Lambda_t^v C_t$<br>$\eta_t = \eta_t^{\text{prior}} + C_t^\top \Lambda_t^v v_t$                                  |

Table 5. Two equivalent views at time  $t$ : moment (KF) vs. canonical (IF). Superscript “prior” denotes the predictive prior; unadorned symbols denote the posterior.

measurement uncertainty.

## D. Theorems and Proofs

**Theorem 1** (Precision Update as a Möbius Transformation). *Let  $\lambda_t$  be the posterior precision at time  $t$  in the diagonal linear-Gaussian model Equation (7)–Equation (9). Define  $\phi_t := k_t^2 \odot \Lambda_t^\vee$ . Then the map  $\lambda_{t-1} \mapsto \lambda_t$  is a linear-fractional (Möbius) transform:*

$$\lambda_t = M_t(\lambda_{t-1}) = \frac{\alpha_t \odot \lambda_{t-1} + \beta_t}{\gamma_t \odot \lambda_{t-1} + \delta_t}, \quad (16)$$

$$M_t = \begin{pmatrix} \alpha_t & \beta_t \\ \gamma_t & \delta_t \end{pmatrix} = \begin{pmatrix} 1 + \bar{p}_t \odot \phi_t & \bar{a}_t^2 \odot \phi_t \\ \bar{p}_t & \bar{a}_t^2 \end{pmatrix}. \quad (17)$$

*Proof of Theorem 1.* Consider the 1D (or diagonal) linear-Gaussian state space model

$$z_t = a_t z_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, p_t), \quad v_t = k_t z_t + \nu_t, \quad \nu_t \sim \mathcal{N}(0, (\Lambda_t^\vee)^{-1}),$$

and define the posterior precision  $\lambda_t := \text{Var}(z_t \mid v_{1:t})^{-1}$ .

**Step 1 (Information-form predict/update).**

$$\text{(predict)} \quad \lambda_t^{\text{prior}} = (a_t^2 \lambda_{t-1}^{-1} + p_t)^{-1} = \frac{\lambda_{t-1}}{a_t^2 + p_t \lambda_{t-1}}, \quad \text{(update)} \quad \lambda_t = \lambda_t^{\text{prior}} + k_t^2 \Lambda_t^\vee.$$

**Step 2 (Single-step recursion and rearrangement).** Eliminate the intermediate  $\lambda_t^{\text{prior}}$  to obtain

$$\lambda_t = \frac{\lambda_{t-1}}{a_t^2 + p_t \lambda_{t-1}} + k_t^2 \Lambda_t^\vee = \frac{(1 + p_t k_t^2 \Lambda_t^\vee) \lambda_{t-1} + a_t^2 k_t^2 \Lambda_t^\vee}{p_t \lambda_{t-1} + a_t^2}.$$

Hence  $\lambda_t$  is a linear-fractional (Möbius) transform of  $\lambda_{t-1}$ ,

$$\lambda_t = M_t(\lambda_{t-1}) = \frac{\alpha_t \lambda_{t-1} + \beta_t}{\gamma_t \lambda_{t-1} + \delta_t}, \quad M_t = \begin{pmatrix} \alpha_t & \beta_t \\ \gamma_t & \delta_t \end{pmatrix} = \begin{pmatrix} 1 + p_t \phi_t & a_t^2 \phi_t \\ p_t & a_t^2 \end{pmatrix},$$

where  $\phi_t := k_t^2 \Lambda_t^\vee$ . Thus, the precision recursion is a Möbius transformation with the stated matrix form. The diagonal multivariate case follows by applying the same scalar derivation to each diagonal element.  $\square$

**Corollary 1.1** (Precision Updates via Parallel Prefix Scan). *Given  $\lambda_0$  and matrices  $\{M_t\}_{t=1}^T$  from Theorem 1, the posterior precisions  $\{\lambda_t\}_{t=1}^T$  can be computed by a parallel prefix scan over  $\{M_t\}$  with  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth.*

*Proof of Corollary 1.1.* By Theorem 1, each precision update is a Möbius transformation:  $\lambda_t = M_t(\lambda_{t-1})$ . Since Möbius transformations compose via matrix multiplication, the precision at time  $t$  can be expressed as

$$\lambda_t = M_t \circ M_{t-1} \circ \cdots \circ M_1(\lambda_0) = (M_t \cdot M_{t-1} \cdots M_1) \cdot \lambda_0,$$

where the composition  $M_{1:t} := \prod_{s=1}^t M_s$  is computed via standard  $2 \times 2$  matrix multiplication. Since matrix multiplication is associative, the product  $M_{1:T}$  can be computed via a parallel prefix scan (Blelloch, 1990) with  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth on  $T$  processors. Once the prefix products  $\{M_{1:t}\}_{t=1}^T$  are available, each precision  $\lambda_t = M_{1:t} \cdot \lambda_0$  is obtained in  $\mathcal{O}(1)$  time, yielding the claimed complexity.  $\square$

**Remark** (Practical Implementation). *In practice, the parallel prefix scan can be implemented efficiently on modern hardware using frameworks such as JAX’s `lax.associative_scan` or PyTorch’s `parallel_primitives`. The  $\mathcal{O}(\log T)$  depth translates to logarithmic wall-clock time on sufficiently parallel hardware, matching the computational efficiency of deterministic SSMS like Mamba while maintaining probabilistic semantics.*

**Theorem 2** (Mean Update as Affine Transformations). *Let  $\eta_t := \lambda_t \odot \mu_t$  be the posterior information mean and  $\Lambda_t^v$  the value precision. Given the precision path  $\{\lambda_t\}$ , the information mean evolves affinely:*

$$\eta_t = \underbrace{(\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1})^{-1} \odot \bar{\mathbf{a}}_t}_{f_t \text{ (history-dependent forget gate)}} \odot \eta_{t-1} + \underbrace{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}_{\text{token evidence}}. \quad (19)$$

*Proof of Theorem 2.* Write the information parameters as  $\eta_t := \Lambda_t \mu_t$  and let  $\Lambda_t^v$  denote the value precision. The information-form measurement update is

$$\Lambda_t = \Lambda_t^{\text{prior}} + k^2 \Lambda_t^v, \quad \eta_t = \eta_t^{\text{prior}} + k \Lambda_t^v v_t,$$

so the observation contribution is affine with coefficient  $k \Lambda_t^v$ .

For the time-prediction of the information mean, use  $\mu_t^{\text{prior}} = a \mu_{t-1}$  and  $\eta_t^{\text{prior}} = \Lambda_t^{\text{prior}} \mu_t^{\text{prior}}$ . Eliminating the means gives

$$\eta_t^{\text{prior}} = \Lambda_t^{\text{prior}} a \mu_{t-1} = \Lambda_t^{\text{prior}} a \Lambda_{t-1}^{-1} \eta_{t-1}.$$

Hence, given the (known) precision path  $\{\Lambda_t, \Lambda_t^{\text{prior}}\}$ ,

$$\eta_t = \underbrace{(\Lambda_t^{\text{prior}} a \Lambda_{t-1}^{-1})}_{f_t} \eta_{t-1} + k \Lambda_t^v v_t, \quad \mu_t = \Lambda_t^{-1} \eta_t.$$

Finally, substituting the information-form predict step  $\Lambda_t^{\text{prior}} = \Lambda_{t-1} / (a^2 + p \Lambda_{t-1})$  (Step 1 of the proof of Theorem 1) eliminates  $\Lambda_t^{\text{prior}}$  from the forget gate,

$$f_t = \Lambda_t^{\text{prior}} a \Lambda_{t-1}^{-1} = \frac{a}{a^2 + p \Lambda_{t-1}} = (a^2 + p \Lambda_{t-1})^{-1} a,$$

which is exactly the forget gate in the statement of Theorem 2. This proves the claimed affine form.  $\square$

**Corollary 2.2** (Posterior-mean recursion as a gated RNN update). *The moment-form posterior mean  $\mu_t = \eta_t \odot \lambda_t$  of the information recursion in Theorem 2 satisfies the gated recurrence*

$$\mu_t = \bar{\mathbf{a}}_t \odot \left( \mathbf{1} - \frac{\mathbf{k}_t^2 \odot \Lambda_t^v}{\lambda_t} \right) \odot \mu_{t-1} + \frac{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}{\lambda_t}, \quad (20)$$

*the diagonal/scalar instance of the KLA state update in Table 3.*

*Proof of Corollary 2.2.* By Theorem 2,  $\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^v v_t$  with forget gate  $f_t = \lambda_t^{\text{prior}} a_t \lambda_{t-1}^{-1}$ . By Step 1 of the proof of Theorem 1, the predicted precision is

$$\lambda_t^{\text{prior}} = \frac{\lambda_{t-1}}{a_t^2 + p_t \lambda_{t-1}} = \rho_t \lambda_{t-1}, \quad \lambda_t = \lambda_t^{\text{prior}} + k_t^2 \Lambda_t^v.$$

Substituting  $\eta_{t-1} = \lambda_{t-1} \mu_{t-1}$  and  $\mu_t = \eta_t / \lambda_t$ , and using  $f_t \lambda_{t-1} = \lambda_t^{\text{prior}} a_t$ ,

$$\mu_t = \frac{\lambda_t^{\text{prior}} a_t \mu_{t-1} + k_t \Lambda_t^v v_t}{\lambda_t}.$$

Since  $\lambda_t^{\text{prior}} = \lambda_t - k_t^2 \Lambda_t^v$ , we have  $\lambda_t^{\text{prior}} / \lambda_t = 1 - k_t^2 \Lambda_t^v / \lambda_t$ , hence

$$\mu_t = a_t \left( 1 - \frac{k_t^2 \Lambda_t^v}{\lambda_t} \right) \mu_{t-1} + \frac{k_t \Lambda_t^v}{\lambda_t} v_t,$$

the claimed gated recurrence. The diagonal multivariate case applies elementwise.  $\square$

**Theorem 3** (Convolutional Form for Deterministic LTI Systems). *Under the conditions: (i) time-invariant dynamics,  $\mathbf{a}_t \equiv \mathbf{a}$ ,  $\mathbf{k}_t \equiv \mathbf{k}$ ; and (ii) deterministic dynamics, process noise  $\mathbf{p}_t = \mathbf{0}$ , both the precision and information mean updates reduce to block-Toeplitz convolutions computable in  $\mathcal{O}(T \log T)$  time via FFT/NTT:*

**Precision updates:**  $\Lambda_t = \sum_{s=0}^t \mathbf{a}^{-2(t-s)} \odot \mathbf{k}^2 \odot \Lambda_s^v$

**Information mean updates:**  $\eta_t = \mathbf{k} \odot \sum_{s=0}^t \mathbf{a}^{t-s} \odot \Lambda_s^v \odot \mathbf{v}_s, \quad \mu_t = \Lambda_t^{-1} \odot \eta_t$

*Proof of Theorem 3.* When  $p_t = 0$  (deterministic dynamics), the Möbius transformation from Theorem 1 simplifies. Specifically, with zero process noise, the precision recursion becomes  $\lambda_t = a^{-2}\lambda_{t-1} + k^2\Lambda_t^v$ . For time-invariant  $a$  and  $k$ , this unrolls to:

$$\lambda_t = a^{-2t}\lambda_0 + k^2 \sum_{s=0}^t a^{-2(t-s)} \Lambda_s^v$$

which is a discrete convolution with kernel  $\kappa[n] = k^2 a^{-2n}$ .

Similarly, from Theorem 2, when  $p_t = 0$  we have  $f_t = a^{-1}$  constant, so the information mean recursion  $\eta_t = a^{-1}\eta_{t-1} + k\Lambda_t^v v_t$  unrolls to:

$$\eta_t = k \sum_{s=0}^t a^{t-s} \Lambda_s^v v_s$$

which is also a convolution with kernel  $h[n] = k a^n$ .

Both convolutions have Toeplitz structure and can be computed via FFT in  $\mathcal{O}(\log T)$  parallel time using the convolution theorem.  $\square$

**Remark (Practical Note).** While theoretically interesting, the convolutional form is primarily relevant for perfectly deterministic systems ( $\mathbf{p} = \mathbf{0}$ ). In practice, KLA uses the more general parallel scan formulation (Corollary 1.1) which handles stochastic dynamics ( $\mathbf{p}_t > \mathbf{0}$ ) efficiently.



**Corollary 2.1** (Mean Updates via Parallel Prefix Scan). *The posterior information means  $\{\eta_t\}_{t=1}^T$  are computable via parallel prefix scan over the affine transformations in Theorem 2 in  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth.*

*Proof of Corollary 2.1.* From Theorem 2, the information mean update has the affine form:

$$\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^\vee v_t$$

where  $f_t = \frac{a_t}{a_t^2 + p_t \lambda_{t-1}}$  (which simplifies to  $a^{-1}$  when  $p_t = 0$ ).

Define the binary associative operator  $\oplus$  on pairs  $(f, b)$  by:

$$(f_2, b_2) \oplus (f_1, b_1) = (f_2 f_1, f_2 b_1 + b_2)$$

This is the standard associative operator for affine transformations.

The composed transformation at time  $t$  is:

$$(f_{1:t}, b_{1:t}) = (f_t, k_t \Lambda_t^\vee v_t) \oplus (f_{t-1}, k_{t-1} \Lambda_{t-1}^\vee v_{t-1}) \oplus \cdots \oplus (f_1, k_1 \Lambda_1^\vee v_1)$$

which gives  $\eta_t = f_{1:t} \eta_0 + b_{1:t}$ .

Since  $\oplus$  is associative, the sequence of compositions can be computed via a parallel prefix scan (Blelloch, 1990) with  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth. Once all information means  $\{\eta_t\}_{t=1}^T$  are computed, the posterior means are obtained by normalising with the precision path from Corollary 1.1:  $\mu_t = \Lambda_t^{-1} \eta_t$ .  $\square$

## E. Additional Empirical Results

### E.1. Ablation on the Importance of Process Noise Parameter

We conduct an ablation that isolates the process-noise parameter by fixing  $\mathbf{p}_t = 0$  (deterministic dynamics) while leaving the rest of KLA unchanged — in particular its *adaptive*, input-dependent observation operator  $\mathbf{k}_t$ . Setting  $\mathbf{p}_t = 0$  freezes the state-dependent factor  $\rho_t$  in the Möbius precision recurrence to a constant and collapses it to a fixed-gate linear update; the recurrence is still a parallel prefix scan with  $\mathcal{O}(T)$  work and  $\mathcal{O}(\log T)$  depth, but its gating is no longer history-dependent. (The stronger convolutional form of Theorem 3, computable via FFT, requires *both*  $\mathbf{p}_t = 0$  and a time-invariant observation operator  $\mathbf{k}_t \equiv \mathbf{k}$ ; with the adaptive  $\mathbf{k}_t$  retained here the ablated system is linear but not time-invariant.) This ablation investigates whether the nonlinear gating supplied by process noise translates into practical benefits.

Table 6 compares full KLA (learnable, time-invariant process noise  $\mathbf{p}$ ) against a deterministic variant with process noise fixed to zero ( $\mathbf{p}_t=0$ , but retaining observation variance  $(\Lambda^\vee)^{-1}$ ), which collapses the Möbius precision recurrence to a fixed-gate linear update. In the single-block MAD-Lab baseline setting the memorisation and recall tasks are essentially saturated for every architecture and show little cross-method variance (Figure 5a); the skills that actually discriminate between models are Compression, Fuzzy Recall, and Selective Copy. Turning process noise off degrades exactly these three hardest tasks, while leaving memorisation, context recall, and noisy recall at  $\sim 100\%$ : Selective Copy and Compression each fall by more than 10 points ( $-14.9$  and  $-12.1$ , respectively) and Fuzzy Recall by 2.5, for a 4.7-point average drop.

Table 6. **Process-noise ablation on the MAD-Lab suite (accuracy %,  $\uparrow$ ).** Full KLA (learnable, time-invariant process noise  $\mathbf{p}$ ) versus a deterministic variant with  $\mathbf{p}_t=0$ , which collapses the Möbius precision recurrence to a fixed-gate linear update. With-noise baselines are the single-block MAD results of Figure 5a. Removing process noise leaves memorisation and exact recall unchanged but degrades the selection-heavy tasks (Selective Copy, Compression).

| KLA variant                      | Compr.   | Mem.    | CR      | NR      | Fuzzy   | Sel. Copy | Avg.    |
|----------------------------------|----------|---------|---------|---------|---------|-----------|---------|
| Learnable $\mathbf{p}$ (full)    | 85.03    | 98.87   | 99.95   | 99.93   | 45.70   | 90.67     | 86.69   |
| $\mathbf{p}_t=0$ (deterministic) | 72.91    | 100.00  | 100.00  | 100.00  | 43.21   | 75.73     | 81.98   |
| $\Delta$ (zero – full)           | $-12.12$ | $+1.13$ | $+0.05$ | $+0.07$ | $-2.49$ | $-14.94$  | $-4.71$ |

These drops move KLA from best-in-class to near the bottom of the field on the very skill it is designed for: on Selective Copy the deterministic variant (75.7) trails every baseline except mLSTM (Figure 5a), erasing KLA’s selection-under-uncertainty advantage. The effect is more pronounced at the larger pretraining scale of Section 5.5, with longer contexts and multiple stacked KLA blocks: there, fixing  $\mathbf{p}_t=0$  frequently destabilises training and diverges to NaNs, as the precision recursion degenerates into a linear scan in which the posterior precision  $\lambda_t$  grows unbounded and the nonlinear, history-dependent gating (Equation (18)) is lost; a non-zero process noise caps the accumulated precision and restores a fading, well-conditioned memory. A theoretical investigation of the stability properties of the system, particularly regarding Riccati controllability and observability, and the role of  $\mathbf{p}_t$  in system stabilisation, represents an interesting direction for future work.

### E.2. Long-Context Associative Recall (Full Sweep)

Multi-Query Associative Recall (MQAR) (Arora et al., 2023) is particularly demanding for fixed-size recurrent architectures, as it directly tests their capacity to store and retrieve multiple key–value associations — a fundamental bottleneck often limited by state dimensionality. Following the Zoology benchmark (Arora et al., 2023), we evaluate on a considerably harder configuration than typically studied: sequence length  $T = 2048$  with vocabulary size  $V = 256$  (matching or exceeding model dimensions; see Table 9).

Figure 6a reveals distinct scaling behaviours across architectures. KLA consistently outperforms Mamba across all dimensions and substantially outperforms GLA, which fails to learn the task under this extreme setting. At lower dimensions ( $d = 64, 128$ ), GDN performs best; its delta-rule mechanism is specifically designed for strong associative recall at limited capacity (Yang et al., 2024), and it operates with a slightly higher effective state size (see Table 12). At  $d = 256$ , KLA outperforms all baselines and reaches near-perfect accuracy ( $> 95\%$ ).

We hypothesise that uncertainty-weighted updates help mitigate state “saturation” under heavy key–value load. Unlike purely additive accumulators, KLA’s fractional-linear (Möbius) precision updates (Equation (18)) adaptively down-weight observations based on posterior uncertainty: when the latent state becomes saturated with many associations, high-confidence

historical information naturally suppresses unreliable new observations. This provides an implicit, learned compression mechanism, analogous to selective pattern storage in high-capacity associative memories (Storkey & Valabregue, 1997), without requiring explicit dimension scaling.

### E.3. Runtime Scaling

**Setup.** We benchmark forward-only runtimes for three KLA implementations: a recurrent (time-stepped) Kalman update, the built-in Torch associative scan, and a custom Triton associative scan kernel. All measurements use Torch 2.9.1 on an NVIDIA A100 (80GB), hidden size 960, and float32 precision. The forward-only pass uses a batch size of 1, simulating test-time prompt processing for a 182 million parameter model—critical for deploying sequence models in production settings.

**Results.** Figure 9 shows that the scan-based implementations scale efficiently with sequence length compared to the recurrent counterparts. These results confirm that information-form filtering can be implemented with the same scan-parallel profile as modern SSM/GLA-style mixers, making KLA practical for long-context prompt processing.

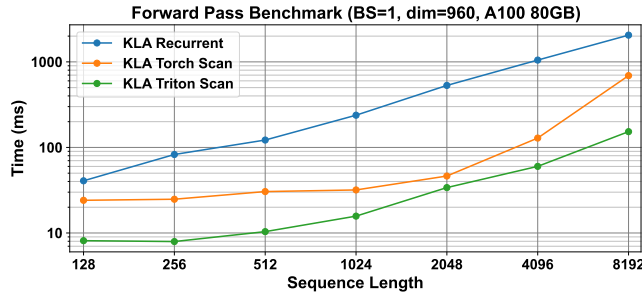


Figure 9. **Forward pass (prompt processing) runtime scaling.** Wall-clock runtime of KLA implementations across sequence lengths during forward-only pass. Torch Scan uses `torch._higher_order_ops.associative_scan`; Triton Scan uses custom forward kernels.

### E.4. Equivalent Attention Matrix

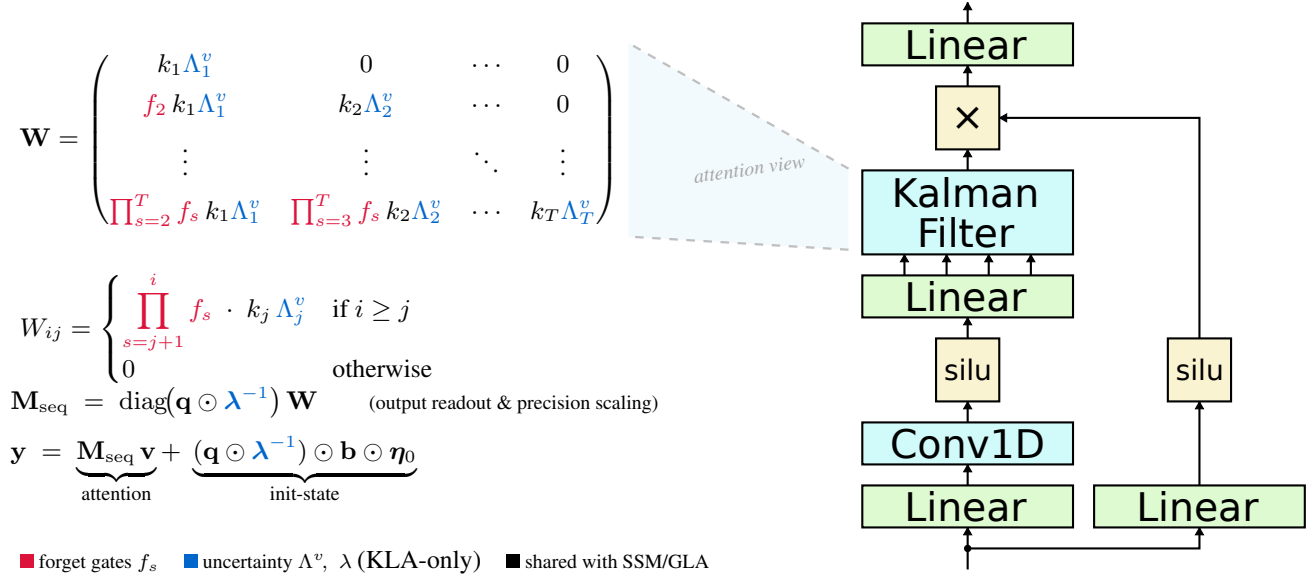
As shown in Theorem 2, the information-mean recurrence  $\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^v v_t$  (Equation (19)) can be unrolled into a lower-triangular attention matrix  $\mathbf{W}$  whose entries are products of **history-dependent forget gates**  $f_s$ , keys  $k_j$ , and **observation precisions**  $\Lambda_j^v$ . Folding in the output readout query  $\mathbf{q}$  and posterior precision scaling  $\lambda^{-1}$  gives the full per-channel sequence transformation  $\mathbf{M}_{\text{seq}} = \text{diag}(\mathbf{q} \odot \lambda^{-1}) \mathbf{W}$ , so that  $\mathbf{y} = \mathbf{M}_{\text{seq}} \mathbf{v}$  (plus init-state terms). Figure 10 visualises this two-step structure.

### E.5. Kalman Attention Map Visualisation

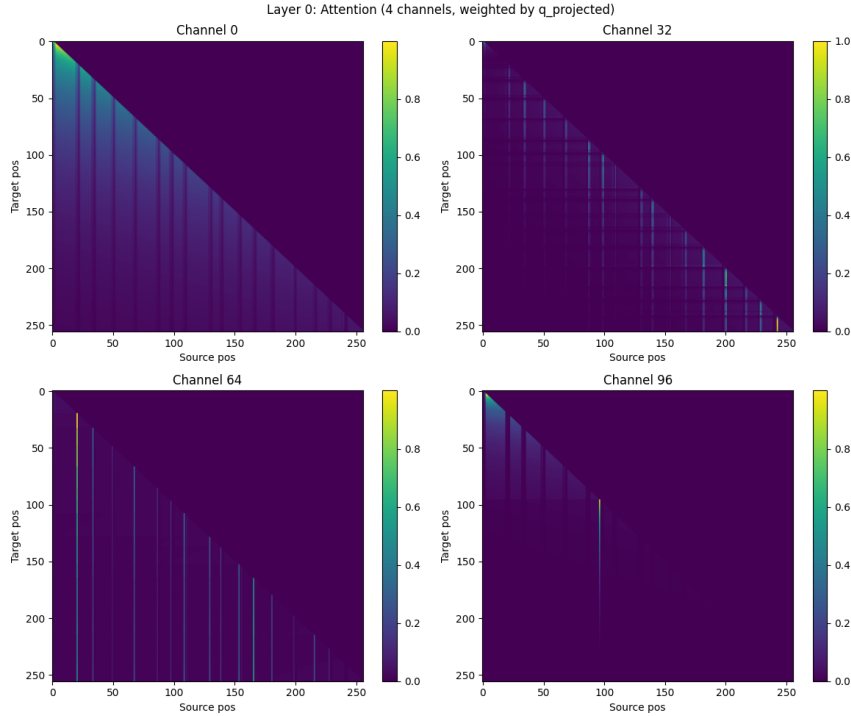
Unrolling the information-mean recurrence equation 19 yields the lower-triangular matrix  $\mathbf{W}$  described in Section E.4; folding in the readout query  $\mathbf{q}_t$  and precision scaling  $\lambda^{-1}$  gives the per-channel sequence transformation  $\mathbf{M}_{\text{seq}} = \text{diag}(\mathbf{q} \odot \lambda^{-1}) \mathbf{W}$  (cf. Figure 10). Because  $\mathbf{M}_{\text{seq}}$  is lower-triangular, it has the same causal structure as a standard attention matrix; we therefore call it the *Kalman Attention Matrix*.

Figures 11 to 13 visualise the Kalman attention patterns learned by KLA on three MADLAB tasks, each showing four randomly chosen channels. The attention weight at position  $(t, s)$  reflects how much the posterior mean at step  $t$  is influenced by the observation at step  $s$ , weighted by the learned precision ratio.

These visualisations demonstrate that KLA learns task-appropriate attention patterns: selective retrieval for copying tasks, channel specialisation for recall, and uniform retention for memorization—all while maintaining the causal structure inherent to the Kalman filter formulation.



**Figure 10. Block architecture and equivalent attention form.** *Left:* The block architecture follows the fused-MLP design of Mamba, with the Kalman Filter as a drop-in replacement for any SSM/Attention primitive. *Right:* Unrolling the information-mean recurrence (Equation (19):  $\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^v v_t$ ) yields a lower-triangular matrix  $\mathbf{W}$  whose entries are products of **history-dependent forget gates**  $f_s$ , keys  $k_j$ , and **observation precisions**  $\Lambda_j^v$ . Applying the output readout query  $q_i$  and **posterior precision**  $\lambda_i^{-1}$  gives the full sequence transformation  $\mathbf{M}_{\text{seq}} = \text{diag}(\mathbf{q} \odot \boldsymbol{\lambda}^{-1}) \mathbf{W}$ —the precision terms being unique to KLA and absent in standard SSMs/GLA.



**Figure 11. Attention maps for Selective Copying** (sequence length 256). The model learns sparse, intermittent vertical bands, attending strongly to a small set of task-relevant positions (mostly copy positions). They either activate or suppress these relevant positions. Differences across channels indicate channel-level specialisation in which positions are retrieved.

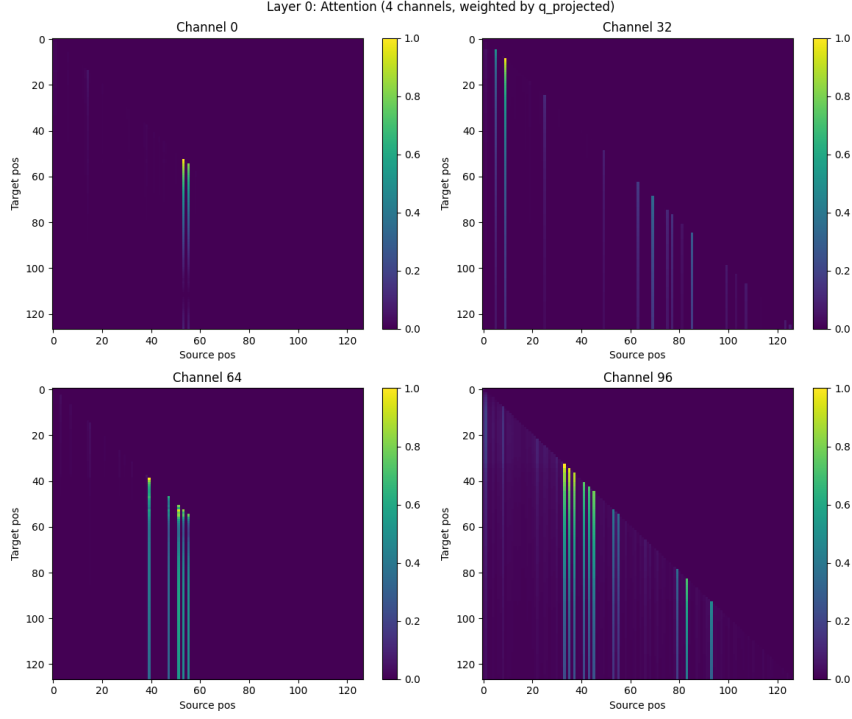


Figure 12. Attention maps for **In-Context Recall** (sequence length 128). The patterns are sparse and “pointer-like”: within each channel, attention concentrates on a small set of source positions (vertical bands) across many target steps, with a mostly low-activation background. Channels show specialisation trends.

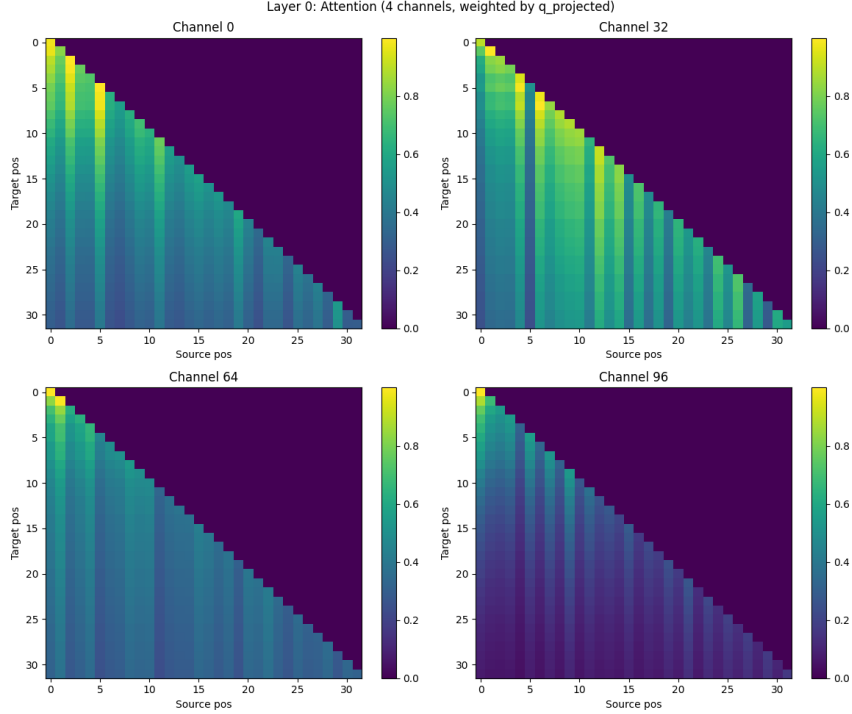


Figure 13. Attention maps for **Memorization** (sequence length 32). The smoother, more gradual decay reflects the task’s requirement to maintain information uniformly across the sequence. All channels show similar patterns, indicating that memorization benefits from redundant, distributed storage rather than selective attention.

Table 7. Mapping from language skills/capabilities to the synthetic probe tasks. Green ✓ indicates the primary probes/tasks for these skills.

| Language Skill / Capability                                                                                                  | In-context (MQAR) | Fuzzy Recall | Noisy Recall | Selective Copy | Compression | Memorization |
|------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------|--------------|----------------|-------------|--------------|
| <b>Prompt-level associative retrieval &amp; in-context learning</b><br>(bindings, multi-query, remapping, basic composition) | ✓                 | ✓            | ✓            | ✓              | ✗           | ✗            |
| <b>Selective attention &amp; noise robustness</b><br>(noise filtering, long-context stability, calibration)                  | ✗                 | ✗            | ✓            | ✓              | ✗           | ✗            |
| <b>Ordered copying &amp; pointer-like control flow</b><br>(order-sensitive working memory)                                   | ✗                 | ✗            | ✗            | ✓              | ✗           | ✗            |
| <b>Information aggregation/bottlenecking</b><br>(token concatenation for downstream decoding)                                | ✗                 | ✗            | ✗            | ✗              | ✓           | ✗            |
| <b>Parametric memory of stable facts</b><br>(weight-based knowledge)                                                         | ✗                 | ✗            | ✗            | ✗              | ✗           | ✓            |

## F. Datasets and Benchmarks Used

### F.1. MAD LM Suite

To evaluate KLA as a language-modelling primitive, we use a standardised suite of six discrete token manipulation tasks. We adopt the MAD framework (Poli et al., 2024), which consolidates synthetic tasks from prior work into a common specification of data generation, splits, and evaluation. Each task probes a distinct capability (associative retrieval, span compositionality, noise robustness, ordered copying, single-token information aggregation, and parametric memory), allowing us to disentangle model strengths and weaknesses across language skills (see Table 7).

We use the baseline configuration from the MAD framework for all six tasks. Table 8 summarises the key parameters for each task.

Table 8. Baseline configurations for the six MAD tasks used in our experiments.

| Parameter        | In-context (MQAR) | Fuzzy Recall | Noisy Recall | Selective Copy | Compression | Memorization |
|------------------|-------------------|--------------|--------------|----------------|-------------|--------------|
| Vocab size       | 16                | 16           | 16           | 16             | 16          | 256          |
| Training seqs    | 12,800            | 12,800       | 12,800       | 12,800         | 12,800      | 256          |
| Sequence length  | 128               | 128          | 128          | 256            | 32          | 32           |
| Noise tokens (%) | —                 | —            | 20%          | —              | —           | —            |
| Key/value split  | 8/8               | 8/8          | 8/8          | —              | —           | 128/128      |

**Additional task-specific settings:** Selective Copy (SC) uses `num_copy=16`; In-Context Recall (CR) and Fuzzy Recall (FR) use `multi_query=True`; FR uses motif sizes  $k = v = 3$ ; Noisy Recall (NR) uses `noise_vocab=16` with noise fraction 0.2.

#### F.1.1. TASK DESCRIPTIONS

**In-context recall (MQAR).** Sequences consist of key-value pairs with separate vocabularies. The model must predict values for keys that appeared earlier in the sequence. Key-value mappings are randomly shuffled between sequences, forcing the model to learn in-context rather than memorising fixed associations.

**Fuzzy in-context recall.** An extension of in-context recall where keys and values are represented by variable-length spans (1–3 tokens). This tests the model’s ability to handle compositional keys and maintain associations across multi-token representations.

**Noisy in-context recall.** Similar to in-context recall, but with 20% of tokens from a separate noise vocabulary randomly inserted. This evaluates the model’s robustness to irrelevant information and selective attention capabilities.

**Selective copying.** Sequences contain random tokens interspersed with special [blank] and [insert] tokens. The model must copy non-special tokens to [insert] positions in order, while learning to ignore tokens near [blank] markers. This tests selective memorisation and order-preserving working memory.



**Compression.** Sequences of random tokens ending with a compression token [c]. The model must compress all sequence information into the representation at the compression token position, such that a fixed two-layer MLP can reconstruct any input token given the compressed representation plus a positional encoding.

**Memorization.** A fixed key-value dictionary is used across all sequences. In each sequence, keys appear with their values masked by [insert] tokens. The model must learn the fixed mappings from the training data, as values never appear in the input during training.

#### F.1.2. STANDARD TRAINING SETUP IN MAD FRAMEWORK

All tasks use the same architectural scaffolding and training procedure. Training hyperparameters are detailed in Section G.

For most tasks, the training objective is standard next-token prediction via log likelihood (cross-entropy loss). The **Compression** task uses a different setup: the model’s output at the compression token position is passed through a separate two-layer MLP decoder (dimensions [240, 120]) along with a positional encoding to reconstruct each token in the sequence. During training, the model learns to compress the entire sequence into a single representation that enables accurate reconstruction.

All models use a single-layer architecture with a model dimension  $d_{\text{model}} = 128$  and an effective state size of 2048.

### F.2. Long-Context MQAR

Table 9 lists the data configuration for the long-context MQAR benchmark task (Arora et al., 2023).

Table 9. Long-Context MQAR data parameters.

| Long-Context Multi-Query Associative Recall (MQAR) |             |            |               |           |
|----------------------------------------------------|-------------|------------|---------------|-----------|
| Setting                                            | Seq. length | Vocab size | Training seqs | Eval seqs |
| CR (hard)                                          | 2,048       | 256        | 12,800        | 1,280     |

## G. Implementation Details

### G.1. Experimental Protocol

**Statistical significance.** For the synthetic and state-tracking experiments we report results averaged over 5 random seeds to ensure statistical reliability, with all model comparisons using the same random seeds to ensure fairness; for these tasks we report the mean performance across seeds. For the A5 state tracking task, we consider a task solved if the model achieves  $\geq 90\%$  accuracy in at least one of the five seeds. The language-modelling pretraining experiments of Section 5.5 are the exception: following standard pretraining practice, each model is trained once owing to its substantially higher compute cost.

**Open-source code.** The complete implementation, including all custom kernels and experimental code, is publicly available at [github.com/vaisakh-shaj/kalman-linear-attention](https://github.com/vaisakh-shaj/kalman-linear-attention) to facilitate reproducibility and future research.

### G.2. Training Hyperparameters

We use default settings unless specifically stated otherwise. Table 10 lists the training configuration used across all experiments.

Table 10. **Training Hyperparameters** (fixed across all models and tasks)

| Hyperparameter          | Value              |
|-------------------------|--------------------|
| Optimizer               | AdamW              |
| Learning rate           | $1 \times 10^{-3}$ |
| Learning rate schedule  | None               |
| Maximum epochs          | 750                |
| Early stopping patience | 70 epochs          |
| Gradient clipping       | 5.0                |
| Weight decay            | 0.0                |
| Precision               | 32-bit float       |

**KLA-specific settings:** We use encoder MLP hidden dimension = 120, decoder MLP dimensions = [240, 120], process noise scale  $\mathbf{p} = 0.01$  (initial value), discretisation timestep range  $\Delta_t \in [0.001, 0.1]$ , causal convolution kernel size = 4, and Monte Carlo samples = 10 (used for KLA probabilistic decoding; see Figure 5a).

### G.3. MAD-Lab Hyperparameters

To ensure fair comparison, all models are configured with equal effective state sizes by adjusting architecture-specific expansion parameters. For each model dimension, we match the total number of parameters used for state representation across architectures. Note that GatedDeltaNet does not allow exact state-size matching due to constraints on the minimum number of heads and the requirement that expanded key and value dimensions be multiples of 16 or 32. Hence, we assign GatedDeltaNet the next-largest feasible value compared to the other baselines. Table 11 shows the dimensional configuration for each architecture.

Table 11. **Model Dimensions** (configured for equal state size  $S = 2048$ )

| Model         | # Blocks | $d_{\text{model}}$ | $d_{\text{state}}$ | Other                           | $S$  |
|---------------|----------|--------------------|--------------------|---------------------------------|------|
| KLA (Ours)    | 1        | 128                | 8                  | expand=1                        | 2048 |
| Mamba         | 1        | 128                | 16                 | expand=1                        | 2048 |
| GatedDeltaNet | 1        | 128                | -                  | $e_k=0.5, e_v=1.0, H=4$         | 2048 |
| GLA           | 1        | 128                | -                  | $e_k=0.5, e_v=1.0, H=4$         | 2048 |
| mLSTM         | 1        | 128                | -                  | $H=16, \text{proj\_factor}=2.0$ | 2048 |

**Batch size:** We use a batch size of 172 for all MAD-Lab experiments.

### G.4. MQAR Hyperparameters

For the MQAR (hard) experiments, we evaluate across three model dimensions with equal state sizes per dimension. All models use 2 layers (repeated blocks). As in the MAD-Lab setup, GatedDeltaNet is given the nearest feasible configuration

(see above). Table 12 lists the configuration for each architecture at each dimension.

Table 12. MQAR Model Configurations (state-matched per dimension)

| Model                       | # Blocks | $d_{\text{model}}$ | $d_{\text{state}}$ | Other                    | State Size |
|-----------------------------|----------|--------------------|--------------------|--------------------------|------------|
| $d = 64$ (batch size = 64)  |          |                    |                    |                          |            |
| KLA                         | 2        | 64                 | 16                 | expand=1                 | 2,048      |
| Mamba                       | 2        | 64                 | 16                 | expand=2                 | 2,048      |
| GatedDeltaNet               | 2        | 64                 | -                  | $e_k=0.75, e_v=1.5, H=2$ | 2,304      |
| GLA                         | 2        | 64                 | -                  | $e_k=1.0, e_v=1.0, H=4$  | 2,048      |
| $d = 128$ (batch size = 32) |          |                    |                    |                          |            |
| KLA                         | 2        | 128                | 16                 | expand=1                 | 4,096      |
| Mamba                       | 2        | 128                | 16                 | expand=2                 | 4,096      |
| GatedDeltaNet               | 2        | 128                | -                  | $e_k=0.75, e_v=1.5, H=4$ | 4,608      |
| GLA                         | 2        | 128                | -                  | $e_k=0.5, e_v=1.0, H=4$  | 4,096      |
| $d = 256$ (batch size = 16) |          |                    |                    |                          |            |
| KLA                         | 2        | 256                | 16                 | expand=1                 | 8,192      |
| Mamba                       | 2        | 256                | 16                 | expand=2                 | 8,192      |
| GatedDeltaNet               | 2        | 256                | -                  | $e_k=0.75, e_v=1.5, H=8$ | 9,216      |
| GLA                         | 2        | 256                | -                  | $e_k=0.25, e_v=1.0, H=4$ | 8,192      |

### G.5. A5 State Tracking Hyperparameters

For the A5 state tracking experiments (Section 5.4), we use  $d_{\text{model}} = 1024$  and  $d_{\text{state}} = 16$  for KLA. We train for up to 500 epochs with early stopping (patience 50), and a learning rate of  $3 \times 10^{-4}$ , and we evaluate over 5 seeds. Values for the baselines are reported as in Merrill et al. (2024).

### G.6. Language-Modelling Pretraining and Evaluation Details

This appendix gives the pretraining and evaluation details for the language-modelling experiments of Section 5.5 (results in Table 4). Our pretraining pipeline - data loading, the AdamW parameter grouping, the  $\mu\text{P}$ -style learning-rate scaling, and the trapezoidal learning-rate schedule - follows the nanochat recipe (Karpathy, 2025), which we adapt to our two academic compute budgets and to KLA’s state-space parameters.

**Compute.** All language-modelling models were trained on NVIDIA GH200 Grace Hopper Superchips on the Isambard-AI National AI Research Resource (AIRR) (McIntosh-Smith et al., 2024).

**Pretraining corpus and objective.** All models are pretrained on FineWeb-Edu (Penedo et al., 2024) with the standard next-token-prediction (cross-entropy) objective in single precision (fp32); only the token-embedding table is stored in bfloat16 for memory, while all computation - including the parallel scan - runs in fp32. We use the nanochat BPE tokenizer (Karpathy, 2025) with a vocabulary of 32,768 tokens, trained on the pretraining corpus. We train at two academic scales: a 45M-parameter model on 1.8B tokens and a 180M-parameter model on 10.9B tokens, in both cases following an approximately Chinchilla-style token budget (Hoffmann et al., 2022).

**Models and baselines.** Both the 45M and 180M models use a 12-layer backbone, and all architectures are parameter-matched at each scale. We compare four standalone mixers - GPT (softmax attention), Mamba, Gated DeltaNet (GDN), and KLA- and the hybrid GPT+KLA, in which *only the final attention layer* of the GPT backbone is replaced by a KLA block (Section 5.5). Baseline state/expansion settings are matched following the conventions in Section G.3; we report the KLA-specific per-scale configuration in Table 13.

**Optimisation.** Following nanochat (Karpathy, 2025), all parameters are optimised with AdamW ( $\beta = (0.8, 0.95)$ ,  $\epsilon = 10^{-10}$ ) and gradient clipping 3.0 (Section 5.6); the Muon optimiser is disabled. Parameters are partitioned into groups - token embeddings, the unembedding (LM head), 2-D hidden weights, 1-D parameters, and the state-space parameters ( $\lambda, \Delta, \mathbf{q}, A$ ) - and the state-space group is trained at  $0.1 \times$  the hidden learning rate with zero weight decay, reflecting its higher sensitivity inside the recurrent scan. Base learning rates are tuned at a reference width  $d_{\text{model}} = 768$  and token batch

size  $B = 2^{19}$ , and transferred to other widths and batch sizes by the  $\mu$ P-style factor  $\sqrt{768/d_{\text{model}}}$  (Yang et al., 2021) and the Adam batch-scaling factor  $\sqrt{B/2^{19}}$ . The learning-rate schedule is constant followed by a linear warmdown over the final 40% of training, with no warmup. The remaining settings are summarised in Table 13.

**Table 13. KLA language-modelling pretraining configurations.** We report KLA-specific settings only; both scales use a 12-layer KLA backbone, and baseline configurations are state-matched following Section G.3. Following nanochat (Karpathy, 2025), all parameters are optimised with AdamW ( $\beta = (0.8, 0.95)$ ,  $\epsilon = 10^{-10}$ ) and Muon is disabled. The per-group AdamW learning rate is the base rate scaled internally by  $\sqrt{768/d_{\text{model}}} \cdot \sqrt{B/2^{19}}$  (with  $B$  the token batch size); the state-space parameters ( $\lambda$ ,  $\Delta$ ,  $\mathbf{q}$ ,  $A$ ) use an additional  $0.1 \times$  multiplier, and token embeddings use a separate base rate of 0.3 (scaled by the same factor).

| KLA hyperparameter                | 45M                        | 180M                       |
|-----------------------------------|----------------------------|----------------------------|
| Layers (depth)                    | 12                         | 12                         |
| $d_{\text{model}}$                | 496                        | 1360                       |
| $d_{\text{state}}$                | 32                         | 32                         |
| Causal conv. kernel size          | 4                          | 4                          |
| Process-noise init $\mathbf{p}$   | 0.01                       | 0.01                       |
| Discretisation range $\Delta_t$   | [0.001, 0.1]               | [0.001, 0.1]               |
| Context length                    | 2048                       | 2048                       |
| Training tokens                   | 1.8B                       | 10.9B                      |
| Tokenizer vocabulary              | 32,768                     | 32,768                     |
| Optimiser                         | AdamW                      | AdamW                      |
| Base learning rate (hidden)       | $4 \times 10^{-3}$         | $4 \times 10^{-3}$         |
| SSM learning-rate multiplier      | 0.1                        | 0.1                        |
| Effective peak LR (hidden)        | $2.5 \times 10^{-3}$       | $3.0 \times 10^{-3}$       |
| Effective peak LR (SSM)           | $2.5 \times 10^{-4}$       | $3.0 \times 10^{-4}$       |
| Embedding learning rate (base)    | 0.3                        | 0.3                        |
| LR schedule                       | constant then linear decay | constant then linear decay |
| Warmup steps                      | 0                          | 0                          |
| Warmdown fraction                 | 0.4                        | 0.4                        |
| Weight decay (2-D hidden/lm_head) | 0.1                        | 0.1                        |
| Weight decay (embed/1-D/SSM)      | 0.0                        | 0.0                        |
| Global batch size (tokens)        | 131,072                    | 524,288                    |
| Gradient clipping                 | 3.0                        | 3.0                        |
| Precision                         | fp32                       | fp32                       |

**Zero-shot evaluation.** We report zero-shot performance with the LM Evaluation Harness (?) on eight commonsense benchmarks: LAMBADA (OpenAI split) (Paperno et al., 2016), HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020), ARC-Easy and ARC-Challenge (Clark et al., 2018), WinoGrande (Sakaguchi et al., 2021), OpenBookQA (Mihaylov et al., 2018), and BoolQ (Clark et al., 2019) (Table 4). We report accuracy (acc) for all tasks except HellaSwag and ARC-Challenge, for which we report length-normalised accuracy ( $\text{acc}_n$ ).